

# 详细设计说明书

项目名称：在线论文文档自动生成工具

版本：V2026.07

|     |                     |
|-----|---------------------|
| 成员： | <u>42411051 李思贤</u> |
| 成员： | <u>42411084 余锦标</u> |
| 成员： | <u>42411094 谢宏涛</u> |
| 成员： | <u>42411116 李佳蔚</u> |
| 成员： | <u>42411134 张宗元</u> |
| 成员： | <u>42411202 张熙伟</u> |

实训课程软件工程文档

2026 年 7 月 16 日

# 目录

|                                             |           |
|---------------------------------------------|-----------|
| 目录                                          | 1         |
| <b>1 数据库详细设计</b>                            | <b>5</b>  |
| 1.1 数据库设计概述                                 | 5         |
| 1.2 学院表 (sys_college)                       | 5         |
| 1.3 用户表 (sys_user)                          | 5         |
| 1.4 论文模板表 (template)                        | 6         |
| 1.5 模板版本表 (template_version)                | 7         |
| 1.6 论文主表 (thesis)                           | 8         |
| 1.7 论文章节表 (thesis_section)                  | 8         |
| 1.8 参考文献表 (thesis_reference)                | 9         |
| 1.9 图片资源表 (thesis_image)                    | 10        |
| 1.10 提交记录表 (submission)                     | 10        |
| 1.11 批注表 (annotation)                       | 10        |
| 1.12 批阅记录表 (review_record)                  | 11        |
| 1.13 章节草稿历史表 (thesis_section_draft_history) | 11        |
| 1.14 索引设计                                   | 12        |
| 1.15 数据库初始化策略                               | 12        |
| <b>2 认证与权限模块详细设计</b>                        | <b>12</b> |
| 2.1 模块概述                                    | 12        |
| 2.2 认证流程设计                                  | 13        |
| 2.2.1 用户注册流程                                | 13        |
| 2.2.2 用户登录流程                                | 13        |
| 2.2.3 用户登出流程                                | 14        |
| 2.3 JWT Token 设计                            | 14        |
| 2.3.1 Token 格式与内容                           | 14        |
| 2.3.2 Token 签名算法                            | 14        |
| 2.3.3 Token 解析与校验                           | 15        |
| 2.4 请求认证过滤器 (JwtFilter)                     | 15        |
| 2.5 角色权限拦截器 (RoleInterceptor)               | 16        |
| 2.6 @RoleRequired 注解                        | 17        |
| 2.7 拦截器注册配置 (WebConfig)                     | 18        |
| 2.8 密码加密方案                                  | 18        |
| 2.9 Redis Token 缓存策略                        | 19        |
| 2.10 统一返回体设计                                | 19        |
| 2.11 全局异常处理                                 | 20        |
| 2.12 安全防护措施                                 | 20        |
| 2.13 认证鉴权全链路时序                              | 21        |

|                                     |           |
|-------------------------------------|-----------|
| <b>3 College 实体详细设计</b>             | <b>21</b> |
| 3.1 数据表结构 . . . . .                 | 21        |
| 3.2 业务规则 . . . . .                  | 21        |
| <b>4 Template 实体详细设计</b>            | <b>22</b> |
| 4.1 数据表结构 . . . . .                 | 22        |
| 4.2 模板类型枚举定义 . . . . .              | 22        |
| 4.3 业务规则 . . . . .                  | 22        |
| <b>5 TemplateVersion 实体详细设计</b>     | <b>23</b> |
| 5.1 数据表结构 . . . . .                 | 23        |
| 5.2 版本号递增算法 . . . . .               | 23        |
| 5.3 版本管理核心流程 . . . . .              | 24        |
| <b>6 Thesis 实体详细设计</b>              | <b>24</b> |
| 6.1 数据表结构 . . . . .                 | 24        |
| 6.2 论文状态转换详细规则 . . . . .            | 25        |
| 6.3 论文创建流程 . . . . .                | 25        |
| <b>7 ThesisSection 实体详细设计</b>       | <b>25</b> |
| 7.1 数据表结构 . . . . .                 | 25        |
| 7.2 自动保存策略 . . . . .                | 25        |
| 7.3 章节树查询实现 . . . . .               | 26        |
| <b>8 核心业务流程图</b>                    | <b>27</b> |
| 8.1 流程一：管理员创建并发布模板 . . . . .        | 27        |
| 8.2 流程二：学生创建论文并撰写 . . . . .         | 27        |
| 8.3 流程三：模板版本升级不影响已有论文 . . . . .     | 27        |
| <b>9 Reference 参考文献实体详细设计</b>       | <b>28</b> |
| 9.1 数据表结构 . . . . .                 | 28        |
| 9.2 文献类型枚举定义 . . . . .              | 28        |
| 9.3 核心接口规范 . . . . .                | 28        |
| 9.4 GB/T 7714 格式化算法详细设计 . . . . .   | 29        |
| <b>10 Image 图片实体详细设计</b>            | <b>31</b> |
| 10.1 数据表结构 . . . . .                | 31        |
| 10.2 图片上传核心流程 . . . . .             | 31        |
| 10.3 上传接口规范 . . . . .               | 32        |
| <b>11 文档导出模块详细设计</b>                | <b>32</b> |
| 11.1 总体架构 . . . . .                 | 32        |
| 11.2 Apache POI DOCX 生成算法 . . . . . | 33        |
| 11.3 LibreOffice PDF 转换 . . . . .   | 34        |

|                                                    |           |
|----------------------------------------------------|-----------|
| 11.4 技术验证结论 . . . . .                              | 35        |
| <b>12 学生端页面详细设计概述</b>                              | <b>35</b> |
| 12.1 页面 1: 登录页 (Login.vue) . . . . .               | 36        |
| 12.1.1 页面布局 . . . . .                              | 36        |
| 12.1.2 控件清单 . . . . .                              | 36        |
| 12.1.3 交互逻辑 . . . . .                              | 36        |
| 12.1.4 输入校验规则 . . . . .                            | 36        |
| 12.2 页面 2: 注册页 (Register.vue) . . . . .            | 37        |
| 12.2.1 页面布局 . . . . .                              | 37        |
| 12.2.2 控件清单 . . . . .                              | 37        |
| 12.2.3 交互逻辑 . . . . .                              | 37        |
| 12.2.4 输入校验规则 . . . . .                            | 37        |
| 12.3 页面 3: 论文列表页 (PaperList.vue) . . . . .         | 38        |
| 12.3.1 页面布局 . . . . .                              | 38        |
| 12.3.2 控件清单 . . . . .                              | 38        |
| 12.3.3 论文卡片子组件设计 . . . . .                         | 38        |
| 12.3.4 交互逻辑 . . . . .                              | 39        |
| 12.4 页面 4: 论文创建向导 (PaperCreate.vue) . . . . .      | 39        |
| 12.4.1 页面布局 . . . . .                              | 39        |
| 12.4.2 控件清单 . . . . .                              | 39        |
| 12.4.3 交互逻辑 . . . . .                              | 39        |
| 12.5 页面 5: 论文编辑器主页面 (PaperEditor.vue) . . . . .    | 40        |
| 12.5.1 页面布局 . . . . .                              | 40        |
| 12.5.2 控件清单 . . . . .                              | 40        |
| 12.5.3 编辑器核心交互流程 . . . . .                         | 41        |
| 12.5.4 章节名称校验规则 . . . . .                          | 41        |
| 12.6 页面 6: 参考文献管理面板 (ReferencePanel.vue) . . . . . | 41        |
| 12.6.1 布局 . . . . .                                | 41        |
| 12.6.2 参考文献表单校验规则 . . . . .                        | 41        |
| 12.6.3 交互逻辑 . . . . .                              | 42        |
| 12.7 页面 7: 导出对话框 (ExportDialog.vue) . . . . .      | 42        |
| 12.7.1 布局 . . . . .                                | 42        |
| 12.7.2 控件清单 . . . . .                              | 42        |
| 12.7.3 交互逻辑 . . . . .                              | 42        |
| 12.8 页面 8: 个人中心页 (Profile.vue) . . . . .           | 43        |
| 12.8.1 布局 . . . . .                                | 43        |
| 12.8.2 控件摘要 . . . . .                              | 43        |
| 12.8.3 校验规则 . . . . .                              | 43        |
| <b>13 全局交互规范</b>                                   | <b>44</b> |
| 13.1 消息提示规范 . . . . .                              | 44        |

|                                                |           |
|------------------------------------------------|-----------|
| 13.2 加载状态规范 . . . . .                          | 44        |
| 13.3 响应式适配断点 . . . . .                         | 44        |
| <b>14 管理员端页面详细设计</b>                           | <b>44</b> |
| 14.1 页面 1: 仪表盘页 (AdminDashboard.vue) . . . . . | 44        |
| 14.1.1 页面布局 . . . . .                          | 44        |
| 14.1.2 控件清单 . . . . .                          | 44        |
| 14.1.3 数据接口 . . . . .                          | 45        |
| 14.2 页面 2: 学院管理页 (CollegeAdmin.vue) . . . . .  | 45        |
| 14.2.1 控件清单 . . . . .                          | 45        |
| 14.2.2 交互逻辑 . . . . .                          | 45        |
| 14.3 页面 3: 模板管理页 (TemplateAdmin.vue) . . . . . | 46        |
| 14.3.1 页面布局 . . . . .                          | 46        |
| 14.3.2 模板卡片组件设计 . . . . .                      | 46        |
| 14.3.3 封面字段配置面板 . . . . .                      | 46        |
| 14.3.4 章节结构配置面板 . . . . .                      | 46        |
| 14.3.5 格式参数配置面板 . . . . .                      | 47        |
| <b>15 教师端页面详细设计</b>                            | <b>47</b> |
| 15.1 页面 1: 批阅任务列表页 (ReviewList.vue) . . . . .  | 47        |
| 15.1.1 页面布局 . . . . .                          | 47        |
| 15.1.2 控件清单 . . . . .                          | 47        |
| 15.1.3 数据接口 . . . . .                          | 47        |
| 15.2 页面 2: 论文批阅页 (ReviewDetail.vue) . . . . .  | 48        |
| 15.2.1 页面布局 . . . . .                          | 48        |
| 15.2.2 章节树面板 . . . . .                         | 48        |
| 15.2.3 论文预览区 . . . . .                         | 48        |
| 15.2.4 批注定位算法 . . . . .                        | 49        |
| 15.2.5 批注面板 . . . . .                          | 49        |
| 15.3 评分对话框详细设计 . . . . .                       | 50        |
| 15.3.1 控件布局 . . . . .                          | 50        |
| 15.3.2 校验规则 . . . . .                          | 50        |
| 15.3.3 提交流程 . . . . .                          | 51        |
| 15.4 前端状态管理 . . . . .                          | 51        |
| <b>16 批阅模块、缓存策略与部署详细设计</b>                     | <b>52</b> |
| 16.1 批阅模块详细设计 . . . . .                        | 52        |
| 16.1.1 数据模型 . . . . .                          | 52        |
| 16.1.2 服务层设计 . . . . .                         | 54        |
| 16.1.3 API 端点设计 . . . . .                      | 56        |
| 16.1.4 批阅流程状态机 . . . . .                       | 56        |
| 16.2 Redis 缓存策略详细设计 . . . . .                  | 57        |
| 16.2.1 Cache-Aside 模式实现 . . . . .              | 57        |

|        |                             |    |
|--------|-----------------------------|----|
| 16.2.2 | 章节草稿自动保存 . . . . .          | 58 |
| 16.2.3 | JWT 令牌管理 . . . . .          | 58 |
| 16.2.4 | API 限流实现 . . . . .          | 59 |
| 16.2.5 | 缓存监控与运维 . . . . .           | 59 |
| 16.3   | 部署详细设计 . . . . .            | 60 |
| 16.3.1 | Dockerfile . . . . .        | 60 |
| 16.3.2 | Docker Compose 编排 . . . . . | 60 |
| 16.3.3 | 关键配置说明 . . . . .            | 62 |
| 16.3.4 | 生产环境补充建议 . . . . .          | 63 |
| 16.3.5 | 部署操作手册 . . . . .            | 63 |

## 版本修订记录

| 版本   | 日期         | 修改人 | 审核 | 修改说明 |
|------|------------|-----|----|------|
| V1.0 | 2026-07-17 | 全组  | 全组 | 初稿完成 |

# 1 数据库详细设计

## 1.1 数据库设计概述

系统数据库采用 PostgreSQL 16 关系型数据库，数据库名称为`thesis_generator`。整体遵循第三范式（3NF）设计，以论文生命周期为主线，围绕用户角色管理、模板配置管理、论文内容编辑、提交批阅流程四大业务域组织数据表结构。以下给出全部数据表的完整 DDL（Data Definition Language）语句及详细字段说明。

## 1.2 学院表（`sys_college`）

学院表用于存储高校下设各学院的基础信息，是整个系统数据组织体系的最上层实体。用户在注册时选择所属学院，模板按学院维度进行分类管理。

Listing 1: `sys_college` 建表语句

```
1 CREATE TABLE IF NOT EXISTS sys_college (  
2     id BIGSERIAL PRIMARY KEY,  
3     name VARCHAR(100) NOT NULL,  
4     code VARCHAR(50) NOT NULL UNIQUE,  
5     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
6     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
7 );
```

- `id`: BIGSERIAL 自增主键，数据库自动分配唯一标识；
- `name`: 学院全称，VARCHAR(100)，不可为空；
- `code`: 学院编号（如 CS、EE、MATH），VARCHAR(50)，唯一约束防止重复；
- `created_at` / `updated_at`: 时间戳字段，记录创建和更新时间。

## 1.3 用户表（`sys_user`）

用户表是系统权限体系的核心，存储全部用户（管理员、学生、教师）的账号信息、加密密码、角色标识和学院归属。

Listing 2: `sys_user` 建表语句

```
1 CREATE TABLE IF NOT EXISTS sys_user (  
2     id BIGSERIAL PRIMARY KEY,  
3     username VARCHAR(50) NOT NULL UNIQUE,  
4     password VARCHAR(255) NOT NULL,  
5     real_name VARCHAR(50) NOT NULL,  
6     role VARCHAR(20) NOT NULL CHECK (role IN ('ADMIN', 'STUDENT', 'TEACHER')),  
7     college_id BIGINT REFERENCES sys_college(id),  
8     enabled BOOLEAN DEFAULT TRUE,
```



```
9      created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ,
10      updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
11 );
```

- **username:** 登录用户名, VARCHAR(50), 唯一约束, 用于登录认证;
- **password:** BCrypt 哈希加密后的密码密文, VARCHAR(255) 以容纳 60 字符的 BCrypt 哈希值;
- **real\\_name:** 用户真实姓名, 用于前端界面显示;
- **role:** 角色枚举, CHECK 约束限定为 ADMIN、STUDENT、TEACHER 三种取值;
- **college\\_id:** 外键引用 sys\\_college.id, 可为空 (管理员可不归属特定学院);
- **enabled:** 账号启用状态, 禁用后该账号无法登录。

角色权限体系设计如表 1 所示:

表 1: 角色权限矩阵

| 功能权限        | ADMIN | STUDENT             | TEACHER             |
|-------------|-------|---------------------|---------------------|
| 用户注册        |       | (仅 STUDENT/TEACHER) | (仅 STUDENT/TEACHER) |
| 模板管理 (CRUD) |       | ×                   | ×                   |
| 学院管理        |       | ×                   | ×                   |
| 创建/编辑论文     | ×     |                     | ×                   |
| 文档导出        | ×     |                     | ×                   |
| 提交论文        | ×     |                     | ×                   |
| 批注/评阅论文     | ×     | ×                   |                     |
| 查看批阅结果      | ×     |                     |                     |

1.4 论文模板表 (template)

模板表存储由管理员创建的论文模板基本信息, 每个模板可关联特定学院, 并设定论文类型。

Listing 3: template 建表语句

```
1 CREATE TABLE IF NOT EXISTS template (
2     id BIGSERIAL PRIMARY KEY,
3     name VARCHAR(200) NOT NULL,
4     type VARCHAR(20) NOT NULL CHECK (type IN ('GRADUATION', 'COURSE', 'PROJECT')),
5     college_id BIGINT REFERENCES sys_college(id),
6     enabled BOOLEAN DEFAULT TRUE,
7     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
8     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
9 );
```

- **name:** 模板名称（如“计算机学院本科毕业论文模板”），VARCHAR(200)；
- **type:** 论文类型，CHECK 约束限定三种取值：GRADUATION（毕业论文）、COURSE（课程论文）、PROJECT（项目论文）；
- **college\\_id:** 外键引用 sys\\_college.id，实现按学院的模板隔离；
- **enabled:** 启用状态，停用的模板不显示给学生创建论文。

## 1.5 模板版本表（template\\_version）

模板版本表是系统灵活性的关键设计。利用 PostgreSQL 特有的 JSONB 类型存储可变结构的配置数据，避免了为不同模板类型设计大量冗余字段的问题。

Listing 4: template\\_version 建表语句

```

1 CREATE TABLE IF NOT EXISTS template_version (
2     id BIGSERIAL PRIMARY KEY,
3     template_id BIGINT NOT NULL REFERENCES template(id) ON DELETE CASCADE,
4     version_number VARCHAR(20) NOT NULL DEFAULT '1.0',
5     is_current BOOLEAN DEFAULT TRUE,
6     cover_fields JSONB NOT NULL DEFAULT '[]',
7     chapter_structure JSONB NOT NULL DEFAULT '[]',
8     format_config JSONB NOT NULL DEFAULT '{}',
9     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
10 );

```

JSONB 配置字段的设计与使用场景：

- **cover\\_fields**（JSON 数组）：定义封面需填写的字段列表和显示顺序。示例：

```

1 [{"key": "title", "label": "论文题目", "required": true},
2  {"key": "author", "label": "作者姓名", "required": true},
3  {"key": "college", "label": "学院", "required": true}]

```

- **chapter\\_structure**（JSON 数组）：定义论文的章节树结构。示例：

```

1 [{"key": "abstract", "title": "摘要", "type": "front"},
2  {"key": "ch1", "title": "绪论", "type": "chapter", "children": [
3      {"key": "ch1_1", "title": "研究背景"},
4      {"key": "ch1_2", "title": "研究意义"}]},
5  {"key": "ref", "title": "参考文献", "type": "back"}]

```

- **format\\_config**（JSON 对象）：定义排版格式参数。示例：

```

1 {"fontFamily": "SimSun", "fontSize": 12, "lineSpacing": 1.5,
2  "marginTop": 2.5, "marginBottom": 2.5,
3  "marginLeft": 3.0, "marginRight": 2.5, "pageSize": "A4"}

```

ON DELETE CASCADE 确保删除模板时其全部版本记录同步删除，维护数据一致性。

## 1.6 论文主表 (thesis)

论文主表记录每篇论文的元信息和生命周期状态。

Listing 5: thesis 建表语句

```

1 CREATE TABLE IF NOT EXISTS thesis (
2     id BIGSERIAL PRIMARY KEY,
3     student_id BIGINT NOT NULL REFERENCES sys_user(id),
4     template_version_id BIGINT REFERENCES template_version(id),
5     title VARCHAR(500) NOT NULL DEFAULT '未命名论文',
6     status VARCHAR(20) NOT NULL DEFAULT 'DRAFT'
7     CHECK (status IN ('DRAFT', 'COMPLETED', 'SUBMITTED',
8                     'REVIEWED', 'RETURNED', 'GENERATING')),
9     is_locked BOOLEAN DEFAULT FALSE,
10    college_id BIGINT REFERENCES sys_college(id),
11    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
12    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
13 );

```

论文状态机流转规则如下，状态转换关系如图 1 所示：

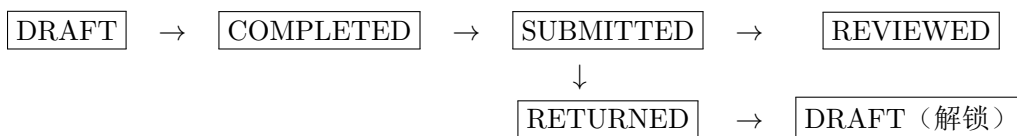


图 1: 论文状态流转图

- DRAFT → COMPLETED: 学生确认论文内容完成；
- COMPLETED → SUBMITTED: 学生提交至教师，is\_locked 置为 TRUE，论文只读；
- SUBMITTED → REVIEWED: 教师完成批阅，给出分数和等级；
- SUBMITTED → RETURNED: 教师退回论文，附带退回原因，is\_locked 置为 FALSE，学生可重新编辑；
- GENERATING: DOCX/PDF 导出任务进行中的中间状态。

## 1.7 论文章节表 (thesis\_section)

章节表存储每篇论文的各章节结构信息和富文本 HTML 内容。

Listing 6: thesis\_section 建表语句

```

1 CREATE TABLE IF NOT EXISTS thesis_section (
2     id BIGSERIAL PRIMARY KEY,
3     thesis_id BIGINT NOT NULL REFERENCES thesis(id) ON DELETE CASCADE,
4     title VARCHAR(300) NOT NULL,

```

```

5      section_key VARCHAR(100) NOT NULL,
6      content TEXT DEFAULT '',
7      draft_content TEXT DEFAULT '',
8      section_type VARCHAR(50) DEFAULT 'chapter',
9      parent_id BIGINT REFERENCES thesis_section(id),
10     sort_order INT DEFAULT 0,
11     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
12     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
13 );

```

- `section\_key`: 章节的业务标识键（如 `ch1`、`ch1_1`、`abstract`），与模板 `version` 的 `chapter\_structure` 中的 `key` 字段对应；
- `content`: 正式保存的章节 HTML 内容；
- `draft\_content`: 自动保存的草稿内容，独立于正式保存，30 秒自动触发保存；
- `section\_type`: 章节类型标识（`chapter/section/subsection/front/back`），用于排版逻辑判断；
- `parent\_id`: 自引用外键，实现多级章节嵌套（父章节 → 子章节）；
- `sort\_order`: 同级章节的排序序号，整数递增，前端按此排序展示；
- `ON DELETE CASCADE`: 删除论文时自动删除全部章节记录。

## 1.8 参考文献表（thesis\_reference）

参考文献表遵循 GB/T 7714-2015 标准的数据元素设计。

**Listing 7:** thesis\_reference 建表语句

```

1 CREATE TABLE IF NOT EXISTS thesis_reference (
2     id BIGSERIAL PRIMARY KEY,
3     thesis_id BIGINT NOT NULL REFERENCES thesis(id) ON DELETE CASCADE,
4     authors VARCHAR(500) NOT NULL,
5     title VARCHAR(500) NOT NULL,
6     journal VARCHAR(300),
7     year VARCHAR(10),
8     volume VARCHAR(50),
9     issue VARCHAR(50),
10    pages VARCHAR(50),
11    doi VARCHAR(200),
12    sort_order INT DEFAULT 0,
13    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
14 );

```

字段设计覆盖了期刊论文、会议论文、学位论文、图书等常见文献类型的核心字段，`doi` 字段支持 DOI 链接跳转。

## 1.9 图片资源表 (thesis\_image)

Listing 8: thesis\_image 建表语句

```

1 CREATE TABLE IF NOT EXISTS thesis_image (
2     id BIGSERIAL PRIMARY KEY,
3     thesis_id BIGINT NOT NULL REFERENCES thesis(id) ON DELETE CASCADE,
4     section_id BIGINT REFERENCES thesis_section(id),
5     original_name VARCHAR(500) NOT NULL,
6     stored_path VARCHAR(500) NOT NULL,
7     url VARCHAR(500) NOT NULL,
8     file_size BIGINT DEFAULT 0,
9     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
10 );

```

- original\\_name: 用户上传时的原始文件名;
- stored\\_path: 服务端文件系统存储路径;
- url: 对外访问 URL (前端 img 标签 src 引用);
- file\\_size: 文件大小 (字节), 用于空间占用统计。

## 1.10 提交记录表 (submission)

Listing 9: submission 建表语句

```

1 CREATE TABLE IF NOT EXISTS submission (
2     id BIGSERIAL PRIMARY KEY,
3     thesis_id BIGINT NOT NULL REFERENCES thesis(id) ON DELETE CASCADE,
4     student_id BIGINT NOT NULL REFERENCES sys_user(id),
5     version_number INT NOT NULL DEFAULT 1,
6     submitted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
7 );

```

每次学生提交论文时新增一条记录, version\\_number 递增, 保留完整的提交历史。

## 1.11 批注表 (annotation)

批注表记录教师在论文指定文字位置添加的修改建议。

Listing 10: annotation 建表语句

```

1 CREATE TABLE IF NOT EXISTS annotation (
2     id BIGSERIAL PRIMARY KEY,
3     thesis_id BIGINT NOT NULL REFERENCES thesis(id) ON DELETE CASCADE,
4     section_id BIGINT NOT NULL REFERENCES thesis_section(id),
5     teacher_id BIGINT NOT NULL REFERENCES sys_user(id),

```

```

6      start_offset INT NOT NULL,
7      text_length INT NOT NULL,
8      selected_text TEXT,
9      content TEXT NOT NULL,
10     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
11 );

```

- `start\_offset`: 选中文字在原 HTML 内容中的起始字符位置 (0-based), 用于精确定位批注锚点;
- `text\_length`: 选中文字的长度, 结合 `start\_offset` 确定完整范围;
- `selected\_text`: 被选中的原文内容 (冗余存储, 便于离线查看批注含义);
- `content`: 教师的批注意见文本。

## 1.12 批阅记录表 (review\_record)

Listing 11: review\_record 建表语句

```

1 CREATE TABLE IF NOT EXISTS review_record (
2     id BIGSERIAL PRIMARY KEY,
3     thesis_id BIGINT NOT NULL REFERENCES thesis(id) ON DELETE CASCADE,
4     teacher_id BIGINT NOT NULL REFERENCES sys_user(id),
5     comment_html TEXT,
6     score INT CHECK (score >= 0 AND score <= 100),
7     grade VARCHAR(10),
8     action VARCHAR(20) NOT NULL CHECK (action IN ('REVIEWED', 'RETURNED')),
9     return_reason TEXT,
10    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
11 );

```

分数 (`score`) 与等级 (`grade`) 的换算规则为:  $\geq 90$  分  $\rightarrow$  优、80-89  $\rightarrow$  良、70-79  $\rightarrow$  中、60-69  $\rightarrow$  及格、 $< 60$   $\rightarrow$  不及格。`action` 区分批阅 (REVIEWED) 和退回 (RETURNED) 两种操作类型。

## 1.13 章节草稿历史表 (thesis\_section\_draft\_history)

Listing 12: thesis\_section\_draft\_history 建表语句

```

1 CREATE TABLE IF NOT EXISTS thesis_section_draft_history (
2     id BIGSERIAL PRIMARY KEY,
3     section_id BIGINT NOT NULL REFERENCES thesis_section(id) ON DELETE CASCADE,
4     draft_content TEXT,
5     saved_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
6 );

```

草稿历史表用于记录每次自动保存的快照, 支持学生回溯到先前的草稿版本。

## 1.14 索引设计

在概要设计中已介绍索引策略，以下给出索引的完整 DDL 语句：

**Listing 13:** 索引创建语句

```

1 CREATE INDEX IF NOT EXISTS idx_user_role ON sys_user(role);
2 CREATE INDEX IF NOT EXISTS idx_template_college ON template(college_id);
3 CREATE INDEX IF NOT EXISTS idx_template_type ON template(type);
4 CREATE INDEX IF NOT EXISTS idx_thesis_student ON thesis(student_id);
5 CREATE INDEX IF NOT EXISTS idx_thesis_status ON thesis(status);
6 CREATE INDEX IF NOT EXISTS idx_section_thesis ON thesis_section(thesis_id);
7 CREATE INDEX IF NOT EXISTS idx_annotation_thesis ON annotation(thesis_id);
8 CREATE INDEX IF NOT EXISTS idx_review_thesis ON review_record(thesis_id);

```

全部索引均采用 B-tree 默认结构。由于当前数据规模为实训项目级别（预计千级用户、万级论文记录），B-tree 索引足以满足所有查询场景的性能需求。若未来数据量增长至百万级，可针对 JSONB 字段（template\_version 的配置数据）添加 GIN 索引以支持配置内容的高效检索。

## 1.15 数据库初始化策略

系统启动时通过 Spring Boot 的 SQL 初始化机制自动执行 schema.sql 脚本。在 application.properties 中配置如下：

**Listing 14:** SQL 初始化配置

```

1 spring.sql.init.mode=always
2 spring.sql.init.schema-locations=classpath:schema.sql
3 spring.jpa.hibernate.ddl-auto=validate

```

ddl-auto=validate 确保 JPA 实体定义与实际数据库表结构保持一致，若存在字段差异则启动报错，防止运行时出现隐式的数据不一致问题。

# 2 认证与权限模块详细设计

## 2.1 模块概述

认证与权限模块（M1）负责系统中全部用户的身份认证和操作授权。模块采用 JWT（JSON Web Token）无状态认证机制配合 Redis 缓存的自研轻量级方案，未引入 Spring Security 全套框架，以降低学习曲线和项目复杂度，同时满足实训项目的安全需求。

模块核心职责包括：

1. **用户认证**：处理注册（创建账号）、登录（签发 Token）、登出（销毁 Token）三项基本认证操作；
2. **Token 管理**：JWT Token 的生成、解析、过期校验，以及 Redis 中的缓存存储；

3. **请求拦截:** 在 API 请求处理管道中拦截所有非公开请求, 校验 Token 有效性并提取用户身份信息;
4. **角色鉴权:** 根据 API 接口声明的所需角色, 校验当前请求用户的角色是否在允许列表中。

## 2.2 认证流程设计

### 2.2.1 用户注册流程

用户注册的完整处理流程如下:

1. 前端收集用户名、密码、真实姓名、角色 (STUDENT 或 TEACHER)、所属学院 ID, 通过 POST 请求发送至 `/api/v1/auth/register`;
2. 请求体经 `@Valid` 注解触发 JSR-303 校验 (用户名 4-50 位、密码 6-100 位、必填字段非空);
3. `AuthController.register()` 接收 `RegisterRequest` DTO, 调用 `AuthService.register()`;
4. `AuthService` 检查用户名是否已存在 (`userRepository.existsByUsername`), 如已存在则抛出 `BusinessException(400, "用户名已存在")`;
5. 校验通过后, 构造 `User` 实体对象: 使用 `BCryptPasswordEncoder.encode()` 对明文密码进行哈希加密, 设置 `enabled=true`, 调用 `userRepository.save()` 持久化;
6. 为新用户生成 JWT Token (`jwtUtil.generateToken`), Token 中包含 `userId`、`username`、`role` 三个 Claim;
7. 将 Token 写入 Redis 缓存 (key 格式: `token:<userId>`), 设置 24 小时过期, 与 JWT 过期时间一致;
8. 返回 `LoginResponse` (`token`、`userId`、`username`、`realName`、`role`)。

### 2.2.2 用户登录流程

登录流程与注册类似但更简化:

1. 前端发送 POST 请求至 `/api/v1/auth/login`, 携带 `LoginRequest` (`username + password`);
2. `AuthService` 根据用户名查询用户 (`userRepository.findByUsername`), 找不到则抛出 `BusinessException(401)`;
3. 检查 `enabled` 状态, 若为 `false` 则抛出 `BusinessException(403, "账号已被禁用")`;
4. 使用 `BCryptPasswordEncoder.matches()` 方法比对明文密码与数据库中的密文;
5. 认证通过后生成 JWT Token 并写入 Redis, 返回 `LoginResponse`。



### 2.2.3 用户登出流程

1. 前端发送 POST 请求至 `/api/v1/auth/logout` (需携带 Token);
2. `AuthController` 从 Request Attribute 中提取 `userId` (由 `JwtFilter` 预先注入);
3. `AuthService` 删除 Redis 中对应的 Token 缓存;
4. Token 本身仍在其有效期内, 但由于后端不再认可该 Token (后续请求在 Redis 中找不到对应记录), 实现了逻辑登出。

## 2.3 JWT Token 设计

### 2.3.1 Token 格式与内容

系统采用 `JWT` 库 (`io.jsonwebtoken 0.12.6`) 实现 JWT Token 的签发和解析。Token 结构为标准的三段式 JWT 格式: `Header.Payload.Signature`。

Payload (Claims) 包含以下字段:

表 2: JWT Token Payload 字段定义

| 字段名                   | 类型                  | 说明                                          |
|-----------------------|---------------------|---------------------------------------------|
| <code>userId</code>   | <code>Long</code>   | 用户 ID, 主键值                                  |
| <code>username</code> | <code>String</code> | 登录用户名                                       |
| <code>role</code>     | <code>String</code> | 角色标识 (ADMIN/STUDENT/TEACHER)                |
| <code>iat</code>      | <code>Date</code>   | 签发时间 (Issued At)                            |
| <code>exp</code>      | <code>Date</code>   | 过期时间 (Expiration), <code>iat + 24 小时</code> |

### 2.3.2 Token 签名算法

Token 签名采用 HMAC-SHA256 对称密钥算法, 密钥通过 `application.properties` 中的 `jwt.secret` 配置项注入。密钥长度建议不少于 256 位 (32 字节), 以确保足够的签名强度。当前开发环境的密钥为占位符, 生产部署前须替换为随机生成的强密钥。

Listing 15: `JwtUtil.generateToken()` 核心逻辑

```
1 public String generateToken(Long userId, String username, String role) {
2     Map<String, Object> claims = new HashMap<>();
3     claims.put("userId", userId);
4     claims.put("username", username);
5     claims.put("role", role);
6
7     return Jwts.builder()
8         .claims(claims)
9         .issuedAt(new Date())
10        .expiration(new Date(System.currentTimeMillis() + expiration))
```

```

11         .signWith(getKey())    // HMAC-SHA256 签名
12         .compact();
13     }

```

### 2.3.3 Token 解析与校验

Token 解析时，JJWT 库自动校验签名和过期时间。若签名不匹配、Token 已过期或格式错误，JwtUtil.parseToken() 抛出相应异常，由 JwtFilter 捕获并统一处理（认证失败时不阻塞请求，仅不注入用户信息，由后续 RoleInterceptor 按需拒绝）。

Listing 16: JwtUtil.parseToken() 核心逻辑

```

1 public Claims parseToken(String token) {
2     return Jwts.parser()
3         .verifyWith(getKey())
4         .build()
5         .parseSignedClaims(token)
6         .getPayload();
7 }

```

## 2.4 请求认证过滤器（JwtFilter）

JwtFilter 是认证机制的核心组件，继承自 Spring Web 的 OncePerRequestFilter 抽象类，确保每次请求仅执行一次过滤逻辑。

Listing 17: JwtFilter.doFilterInternal() 核心逻辑

```

1 @Override
2 protected void doFilterInternal(HttpServletRequest request,
3     HttpServletResponse response, FilterChain chain)
4     throws ServletException, IOException {
5
6     String path = request.getRequestURI();
7
8     // 放行公开端点
9     if (path.startsWith("/api/v1/auth/") ||
10         path.contains("/doc.html") ||
11         path.contains("/v3/api-docs") ||
12         path.contains("/swagger") ||
13         path.contains("/webjars")) {
14         chain.doFilter(request, response);
15         return;
16     }
17
18     // 解析Token并注入用户信息
19     String header = request.getHeader("Authorization");
20     if (header != null && header.startsWith("Bearer ")) {

```

```

21     String token = header.substring(7);
22     try {
23         if (!jwtUtil.isTokenExpired(token)) {
24             Claims claims = jwtUtil.parseToken(token);
25             request.setAttribute("userId",
26                 claims.get("userId", Long.class));
27             request.setAttribute("username",
28                 claims.get("username", String.class));
29             request.setAttribute("role",
30                 claims.get("role", String.class));
31         }
32     } catch (Exception ignored) {
33         // Token无效时不注入属性，后续鉴权拦截器按需拒绝
34     }
35 }
36
37 chain.doFilter(request, response);
38 }

```

关键设计决策说明：

- **不阻断未认证请求：**Filter 不在缺少 Token 或 Token 无效时直接返回 401，而是将请求继续传递给后续拦截器。这样设计的好处是：若后续 Controller 方法未标注 `@RoleRequired`，则允许未登录用户访问（为未来可能的公开接口预留灵活性）；
- **白名单路径：**注册、登录、API 文档（Swagger/Knife4j）相关路径被明确排除在认证拦截之外；
- **无状态设计：**Filter 不查询数据库，仅依赖 JWT Token 自包含的用户信息完成身份识别，减轻数据库压力。

## 2.5 角色权限拦截器（RoleInterceptor）

RoleInterceptor 实现 Spring MVC 的 `HandlerInterceptor` 接口，在 Controller 方法执行前校验用户角色。

Listing 18: RoleInterceptor.preHandle() 核心逻辑

```

1  @Override
2  public boolean preHandle(HttpServletRequest request,
3                          HttpServletResponse response, Object handler) {
4
5      if (!(handler instanceof HandlerMethod hm)) {
6          return true;
7      }
8
9      // 获取方法上的 @RoleRequired 注解
10     RoleRequired annotation = hm.getMethodAnnotation(RoleRequired.class);

```

```

11     if (annotation == null) {
12         annotation = hm.getBeanType().getAnnotation(RoleRequired.class);
13     }
14     if (annotation == null) {
15         return true; // 无注解则放行
16     }
17
18     String role = (String) request.getAttribute("role");
19     if (role == null) {
20         throw new BusinessException(401, "请先登录");
21     }
22
23     List<String> allowed = Arrays.asList(annotation.value());
24     if (!allowed.contains(role)) {
25         throw new BusinessException(403,
26             "权限不足, 需要角色: " + String.join(", ", allowed));
27     }
28
29     return true;
30 }

```

拦截器的工作流程如下:

1. 判断当前 handler 是否为 Controller 方法 (HandlerMethod), 非方法类型 (如静态资源请求) 直接放行;
2. 优先从方法上获取 @RoleRequired 注解, 若不存在则从 Controller 类上获取, 两级均无则放行 (无权限要求);
3. 从 Request Attribute 中取出 role 值 (由 JwtFilter 预先注入), 若为 null 则用户未登录, 抛出 BusinessException(401);
4. 校验当前用户角色是否在 @RoleRequired 声明的允许角色列表中, 不在则抛出 BusinessException(403);
5. 校验通过后返回 true, 请求继续进入 Controller 方法处理。

## 2.6 @RoleRequired 注解

自定义注解 @RoleRequired 用于在 Controller 类或方法级别声明接口所需的角色权限。

Listing 19: @RoleRequired 注解定义

```

1 @Target({ElementType.METHOD, ElementType.TYPE})
2 @Retention(RetentionPolicy.RUNTIME)
3 public @interface RoleRequired {
4     String[] value() default {};
5 }

```

典型使用方式:

- **类级别声明:** 在 Controller 类上标注 @RoleRequired, 该类下所有方法均需指定角色才能访问。例如管理员 Controller 标注@RoleRequired("ADMIN");
- **方法级别声明:** 在特定方法上标注, 覆盖类级别设置。例如在学生 Controller 中, 大部分方法需要 STUDENT 角色, 但某个查询接口允许 TEACHER 查看, 则在方法上标注@RoleRequired({"STUDENT", "TEACHER"});

## 2.7 拦截器注册配置 (WebConfig)

WebConfig 实现 WebMvcConfigurer 接口, 将 RoleInterceptor 注册到 Spring MVC 拦截器链中。

Listing 20: WebConfig 拦截器注册

```

1 @Configuration
2 @RequiredArgsConstructor
3 public class WebConfig implements WebMvcConfigurer {
4
5     private final RoleInterceptor roleInterceptor;
6
7     @Override
8     public void addInterceptors(InterceptorRegistry registry) {
9         registry.addInterceptor(roleInterceptor)
10             .addPathPatterns("/api/**")
11             .excludePathPatterns(
12                 "/api/v1/auth/**", // 认证接口公开
13                 "/doc.html", // Knife4j 文档页
14                 "/v3/api-docs/**", // OpenAPI JSON
15                 "/swagger/**", // Swagger 静态资源
16                 "/webjars/**" // Webjars 资源
17             );
18     }
19 }
```

关键配置说明:

- `addPathPatterns("/api/**")`: 拦截所有 API 请求;
- `excludePathPatterns(...)`: 排除认证接口和 API 文档相关路径, 这些路径在 JwtFilter 中同样被放行, 形成双重保障。

## 2.8 密码加密方案

系统采用 BCrypt 单向哈希算法存储用户密码。BCrypt 是专门为密码哈希设计的算法, 具有以下优势:

- **内置盐值:**每次加密自动生成随机盐值并嵌入哈希结果中(格式:\$2a\$10\$<22位盐值><31位哈希值>), 相同密码每次加密结果不同, 有效防御彩虹表攻击;
- **计算代价可调:**work factor 参数(示例为 10)表示  $2^{10}$  轮哈希迭代, 可根据硬件性能调整。当前默认值 10 在保证安全性的同时, 单次加密耗时约 100ms, 对登录体验影响可忽略;
- **不可逆:**BCrypt 是单向哈希, 无法从密文反推明文密码。

AuthService 中直接创建 BCryptPasswordEncoder 实例:

```
1 private final BCryptPasswordEncoder passwordEncoder =
2     new BCryptPasswordEncoder();
```

注册时调用 `passwordEncoder.encode(rawPassword)` 生成密文存储;登录时调用 `passwordEncoder.matches(r` 比对密码。

## 2.9 Redis Token 缓存策略

Token 在签发的同时写入 Redis 缓存, key 设计为 "token:" + `userId`, value 为 Token 字符串本身。设置过期时间为 24 小时, 与 JWT Token 的过期时间保持一致。

Redis Token 缓存的作用:

- **登出管理:**前端登出时, 后端删除 Redis 中的 Token 记录, 使得该 Token 不再被认可(即便 JWT 本身未过期);
- **单点登录控制:**可选扩展——同一 `userId` 的 Token 写入 Redis 时覆盖旧值, 天然实现了同一账号后续登录踢掉前一次登录的效果;
- **状态查询:**管理员可通过 Redis 中是否存在某用户的 Token 来判断其在线状态。

RedisTemplate 的序列化配置(RedisConfig)采用 String 序列化 key、GenericJackson2JsonRedisSerializer 序列化 value, 支持对象数据的 JSON 序列化存储, 同时保持 Redis 客户端中的 key 可读性。

## 2.10 统一返回体设计

所有 API 响应均通过 `Result<T>` 泛型类统一封装, 结构如下:

**Listing 21:** Result.java 统一返回体

```
1 @Data
2 @NoArgsConstructor
3 @AllArgsConstructor
4 public class Result<T> {
5     private int code;           // 业务状态码
6     private String message;     // 提示信息
7     private T data;             // 响应数据 (泛型)
8 }
```

```
9     public static <T> Result<T> ok(T data) {
10         return new Result<>(200, "success", data);
11     }
12
13     public static <T> Result<T> error(int code, String message) {
14         return new Result<>(code, message, null);
15     }
16 }
```

状态码约定：

- 200：请求成功；
- 400：参数校验失败；
- 401：未登录或 Token 无效；
- 403：角色权限不足；
- 500：服务端内部错误。

## 2.11 全局异常处理

GlobalExceptionHandler 通过 @RestControllerAdvice 注解拦截 Controller 层抛出的异常，转换为统一返回体格式：

- **BusinessException**：业务异常，提取异常中的 code 和 message 构造 Result 返回；
- **MethodArgumentNotValidException**：JSR-303 参数校验异常，汇总所有字段级别的错误信息，以分号分隔返回；
- **Exception**：兜底异常处理，记录完整堆栈日志，向前端返回 500 错误。

该机制确保所有异常（包括认证鉴权模块抛出的 BusinessException）都能以统一的 JSON 格式返回前端，前端只需关注 Result 中的 code 和 message 字段即可完成统一的错误处理逻辑。

## 2.12 安全防护措施

认证权限模块在设计中考虑了以下安全防护点：

1. **密码不落盘**：密码仅以 BCrypt 密文形式存储在数据库中，日志中不打印密码参数（AuthService 中密码字段仅参与比对，不输出日志）；
2. **Token 时效性**：JWT Token 有效期 24 小时，过期后必须重新登录，降低 Token 泄露后的安全风险窗口；

3. **接口级权限控制**: 每个需要权限的 API 接口通过 `@RoleRequired` 明确声明角色要求, 防止水平越权 (学生调用教师接口) 和垂直越权 (普通用户调用管理员接口);
4. **注册角色限制**: `AuthService.register()` 方法中硬编码禁止注册 ADMIN 角色, 管理员账号仅由 DBA 在数据库层面手动创建, 防止恶意注册管理员;
5. **账号禁用机制**: `enabled` 字段支持管理员禁用违规或异常账号, 禁用账号在登录时被明确拒绝 (返回 403 禁用提示, 而非模糊的 401 认证失败)。

## 2.13 认证鉴权全链路时序

用户从登录到访问受保护 API 的完整认证鉴权链路如下:

1. 用户通过 `POST /api/v1/auth/login` 提交用户名和密码;
2. `AuthService` 验证用户名密码, 通过后生成 JWT Token, 写入 Redis, 返回 `LoginResponse` (包含 Token);
3. 前端将 Token 存储在 `localStorage` 中, 并设置 Axios 请求拦截器——每次 HTTP 请求自动在 header 中注入 `Authorization: Bearer <token>`;
4. 用户访问受保护的 API (如 `GET /api/v1/thesis/list`):
  - (a) `JwtFilter` 从 header 提取 Token, 校验签名和过期时间, 将 `userId/username/role` 注入 Request Attribute;
  - (b) `RoleInterceptor` 检查目标 Controller 方法上的 `@RoleRequired` 注解, 校验当前用户角色是否在允许列表中;
  - (c) 鉴权通过后, Controller 方法通过 `request.getAttribute("userId")` 获取当前操作用户 ID, 确保数据隔离 (学生只能操作自己的论文, 教师只能查看自己学院的论文等)。
5. 用户登出时调用 `POST /api/v1/auth/logout`, 服务端删除 Redis 中的 Token 记录, 前端清空 `localStorage` 中的 Token。

该链路的设计确保了从认证到鉴权的完整闭环, 每个环节都有明确的职责边界和异常处理策略, 构成了系统安全体系的基石。

## 3 College 实体详细设计

### 3.1 数据表结构

### 3.2 业务规则

1. 学院代码 (`code`) 创建后不可修改, 确保系统内部标识的稳定性。



表 3: sys\_college 表结构

| 字段名        | 类型           | 约束                 | 默认值   | 说明         |
|------------|--------------|--------------------|-------|------------|
| id         | BIGINT       | PK, AUTO_INCREMENT | —     | 主键         |
| name       | VARCHAR(100) | NOT NULL           | —     | 学院名称       |
| code       | VARCHAR(50)  | NOT NULL, UNIQUE   | —     | 学院代码, 全局唯一 |
| created_at | DATETIME     | —                  | NOW() | 创建时间       |
| updated_at | DATETIME     | —                  | NOW() | 更新时间       |

2. 新增学院时校验 code 唯一性: 若已存在相同 code 返回 409 Conflict。
3. 删除学院前执行关联检查: 查询 template 表 WHERE college\_id = ? AND enabled = TRUE; 查询 thesis 表 WHERE college\_id = ? AND status != 'COMPLETED'。任意检查返回记录数 > 0 则返回错误并阻止删除。
4. 学院列表接口不分页 (数据量有限), 按 name 字段升序返回全部记录。

## 4 Template 实体详细设计

### 4.1 数据表结构

表 4: template 表结构

| 字段名        | 类型           | 约束                            | 默认值   | 说明              |
|------------|--------------|-------------------------------|-------|-----------------|
| id         | BIGINT       | PK, AUTO_INCREMENT            | —     | 主键              |
| name       | VARCHAR(200) | NOT NULL                      | —     | 模板名称            |
| type       | VARCHAR(20)  | NOT NULL                      | —     | 模板类型枚举          |
| college_id | BIGINT       | FK → sys_college.id, NULLABLE | NULL  | 适用学院, NULL 表示通用 |
| enabled    | BOOLEAN      | NOT NULL                      | TRUE  | 启用状态            |
| created_at | DATETIME     | —                             | NOW() | 创建时间            |
| updated_at | DATETIME     | —                             | NOW() | 更新时间            |

### 4.2 模板类型枚举定义

### 4.3 业务规则

1. 创建模板时, college\_id 可为 NULL (通用模板) 或指定学院 ID。
2. 模板列表查询支持 type 和 college\_id 可选筛选参数, 按 updated\_at 降序排列。
3. 启用/禁用: enabled 设为 false 时, 新论文不可选用该模板, 但已使用该模板的论文不受影响。

表 5: Template.type 枚举值

| 枚举值        | 中文名称   | 模板特征                      |
|------------|--------|---------------------------|
| GRADUATION | 毕业设计论文 | 含封面、中英文摘要、目录、多章正文、致谢、参考文献 |
| COURSE     | 课程论文   | 结构简化, 不含中英文摘要和致谢, 一般为单章结构 |
| PROJECT    | 项目报告   | 含项目背景、需求分析、设计方案、实现过程、测试结论 |

4. 删除模板前检查 thesis 表 WHERE template\_version\_id IN (SELECT id FROM template\_version WHERE template\_id = ?) AND status IN ('DRAFT','SUBMITTED','REVIEWED','RETURNED','GENERATED') 存在记录则阻止删除。

## 5 TemplateVersion 实体详细设计

### 5.1 数据表结构

表 6: template\_version 表结构

| 字段名               | 类型          | 约束                         | 默认值   | 说明                   |
|-------------------|-------------|----------------------------|-------|----------------------|
| id                | BIGINT      | PK, AUTO_INCREMENT         | —     | 主键                   |
| template_id       | BIGINT      | FK → template.id, NOT NULL | —     | 所属模板                 |
| version_number    | VARCHAR(20) | NOT NULL                   | '1.0' | 版本号字符串               |
| is_current        | BOOLEAN     | NOT NULL                   | TRUE  | 是否当前生效版本             |
| cover_fields      | TEXT        | —                          | '[]'  | 封面字段配置 (JSON Array)  |
| chapter_structure | TEXT        | —                          | '[]'  | 章节结构配置 (JSON Array)  |
| format_config     | TEXT        | —                          | '{}'  | 排版格式配置 (JSON Object) |
| created_at        | DATETIME    | —                          | NOW() | 创建时间                 |

### 5.2 版本号递增算法

版本号格式为 MAJOR.MINOR (如 1.0、2.3)。

```

1 public String incrementVersion(String currentVersion) {
2     String[] parts = currentVersion.split("\\.");
3     int major = Integer.parseInt(parts[0]);
4     int minor = Integer.parseInt(parts[1]);
5     if (minor < 9) {
6         return major + "." + (minor + 1); // 1.0 -> 1.1
7     } else {

```

```

8         return (major + 1) + ".0";           // 1.9 -> 2.0
9     }
10 }

```

### 5.3 版本管理核心流程

1. **创建模板:**创建 Template 记录的同时,自动创建首条 TemplateVersion 记录(version\_number="1.0", is\_current=true, 三个 JSON 字段初始化为空数组/空对象)。
2. **编辑并保存配置:** 管理员修改 JSON 配置后点击保存, 系统执行: 将当前 is\_current 版本置为 false; 调用 incrementVersion() 计算新版本号; 创建新 TemplateVersion 记录, is\_current=true。
3. **版本回滚:** 管理员选择历史版本点击"激活", 系统执行: UPDATE template\_version SET is\_current = false WHERE template\_id = ? AND is\_current = true; UPDATE template\_version SET is\_current = true WHERE id = ?。回滚不影响已绑定该模板历史版本的论文。
4. **差异对比:** 前端请求两个版本 id, 后端返回两个版本的三个 JSON 字段。前端使用 JSON diff 算法逐字段对比, 以绿色标记新增、红色标记删除、灰色标记未变更。

## 6 Thesis 实体详细设计

### 6.1 数据表结构

表 7: thesis 表结构

| 字段名                 | 类型           | 约束                       | 默认值     | 说明        |
|---------------------|--------------|--------------------------|---------|-----------|
| id                  | BIGINT       | PK, AUTO_INCREMENT       | —       | 主键        |
| student_id          | BIGINT       | FK → user.id, NOT NULL   | —       | 所属学生      |
| template_version_id | BIGINT       | FK → template_version.id | —       | 使用的模板版本快照 |
| title               | VARCHAR(500) | NOT NULL                 | '未命名论文' | 论文标题      |
| status              | VARCHAR(20)  | NOT NULL                 | 'DRAFT' | 论文状态      |
| is_locked           | BOOLEAN      | NOT NULL                 | FALSE   | 是否锁定      |
| college_id          | BIGINT       | FK → sys_college.id      | —       | 所属学院      |
| created_at          | DATETIME     | —                        | NOW()   | 创建时间      |
| updated_at          | DATETIME     | —                        | NOW()   | 更新时间      |

表 8: 论文状态转换详细规则

| 当前状态       | 目标状态       | 前置条件            | 后置动作                        |
|------------|------------|-----------------|-----------------------------|
| DRAFT      | SUBMITTED  | 所有章节 content 非空 | is_locked = true; 记录提交时间    |
| DRAFT      | GENERATING | 所有章节 content 非空 | is_locked = true; 提交异步导出任务  |
| SUBMITTED  | REVIEWED   | 教师完成批注          | 通知学生                        |
| SUBMITTED  | RETURNED   | 教师退回            | is_locked = false; 通知学生修改   |
| RETURNED   | DRAFT      | 学生进入编辑页         | 自动切换（无额外操作）                 |
| GENERATING | COMPLETED  | 导出任务成功          | 记录导出文件路径; 通知学生下载            |
| GENERATING | DRAFT      | 导出任务失败          | is_locked = false; 通知学生失败原因 |

## 6.2 论文状态转换详细规则

## 6.3 论文创建流程

1. 学生 POST /api/theses, 请求体包含 title、templateVersionId、collegeId。
2. 后端校验: title 非空, templateVersionId 对应版本存在, collegeId 对应学院存在。
3. 创建 Thesis 记录 (status=DRAFT)。
4. 查询 TemplateVersion 的 chapter\_structure JSON, 递归遍历:
  - (a) 对每个 type 不是"auto"的节点, 创建 ThesisSection 记录 (thesis\_id= 新论文 id, section\_key= 节点的 key, title= 节点的 title, section\_type= 节点的 type, parent\_id= 父章节 id, sort\_order= 遍历顺序)。
  - (b) type 为"auto"的节点 (如目录) 不创建 ThesisSection 记录。
5. 返回创建成功的 Thesis 对象。

## 7 ThesisSection 实体详细设计

### 7.1 数据表结构

### 7.2 自动保存策略

- 前端编辑器每 30 秒触发 POST /api/sections/{id}/auto-save, 请求体中仅包含 { "draftContent": "..."}。

表 9: thesis\_section 表结构

| 字段名           | 类型           | 约束                       | 默认值       | 说明                          |
|---------------|--------------|--------------------------|-----------|-----------------------------|
| id            | BIGINT       | PK, AUTO_INCREMENT       | —         | 主键                          |
| thesis_id     | BIGINT       | FK → thesis.id, NOT NULL | —         | 所属论文                        |
| title         | VARCHAR(300) | NOT NULL                 | —         | 章节标题                        |
| section_key   | VARCHAR(100) | NOT NULL                 | —         | 对应模板 chapterStructure 的 key |
| content       | TEXT         | —                        | ”         | 正式内容（富文本 HTML）              |
| draft_content | TEXT         | —                        | ”         | 草稿内容（自动保存）                  |
| section_type  | VARCHAR(50)  | —                        | 'chapter' | 章节类型                        |
| parent_id     | BIGINT       | —                        | NULL      | 父章节 ID（自引用外键，NULL 为根节点）     |
| sort_order    | INT          | —                        | 0         | 同层级排序序号                     |
| created_at    | DATETIME     | —                        | NOW()     | 创建时间                        |
| updated_at    | DATETIME     | —                        | NOW()     | 更新时间                        |

- 后端仅更新 draft\_content 字段，不影响 content 字段，不触发论文 updated\_at 更新。
- 学生手动点击“保存”时，PUT /api/sections/{id}，请求体 { "content": "..."}，将正式内容写入 content 字段并更新论文 updated\_at。
- 前端加载章节内容时优先级：draft\_content 非空时展示 draft\_content（提示“恢复未保存内容”），否则展示 content。

### 7.3 章节树查询实现

获取论文全部章节树的 SQL 查询：

```

1 // Repository 层
2 @Query("SELECT s FROM ThesisSection s WHERE s.thesisId = :thesisId ORDER BY s.
   sortOrder ASC")
3 List<ThesisSection> findAllByThesisId(@Param("thesisId") Long thesisId);
4
5 // Service 层 —— 构建树形结构
6 public List<SectionTreeNode> buildTree(Long thesisId) {
7     List<ThesisSection> all = repository.findAllByThesisId(thesisId);
8     Map<Long, List<ThesisSection>> parentMap = all.stream()
9         .filter(s -> s.getParentId() != null)
10        .collect(Collectors.groupingBy(ThesisSection::getParentId));
11    return all.stream()
12        .filter(s -> s.getParentId() == null) // 根节点
13        .map(root -> buildNode(root, parentMap))
14        .collect(Collectors.toList());
15 }
16
17 private SectionTreeNode buildNode(ThesisSection section,

```

```
18     Map<Long, List<ThesisSection>> parentMap) {
19     SectionTreeNode node = new SectionTreeNode(section);
20     List<ThesisSection> children = parentMap.getOrDefault(section.getId(), List.of
21         ());
22     node.setChildren(children.stream()
23         .map(child -> buildNode(child, parentMap))
24         .collect(Collectors.toList()));
25     return node;
26 }
```

## 8 核心业务流程图

### 8.1 流程一：管理员创建并发布模板

1. 管理员 POST /api/templates, 创建模板 → Template + TemplateVersion(V1.0) 入库。
2. 管理员进入版本编辑页, 配置 coverFields (拖拽封面元素)、chapterStructure (增删章节节点)、formatConfig (设置字体页边距)。
3. 管理员点击”预览”: 系统调用预览服务, 用示例数据 + 当前 JSON 配置渲染 PDF → 浏览器新标签页展示。
4. 管理员确认无误, 点击”保存并发布”: 系统递增版本号, 创建新 TemplateVersion 记录 → 学生端模板列表立即可见。

### 8.2 流程二：学生创建论文并撰写

1. 学生 POST /api/theses → 系统创建 Thesis(DRAFT) + 按模板 chapterStructure 批量创建 ThesisSection 记录。
2. 前端加载编辑器: 左侧渲染章节树 (GET /api/theses/{id}/sections), 右侧渲染富文本编辑器。
3. 学生切换章节 → 前端 GET 对应 section 的 content/draftContent → 加载到编辑器。
4. 学生编辑内容 → 每 30 秒 auto-save draftContent → 手动保存时写 content。
5. 全部章节完成后, 学生点击”提交” → 校验所有 content 非空 → status 变为 SUBMITTED → is\_locked=true。

### 8.3 流程三：模板版本升级不影响已有论文

1. 管理员修改模板 JSON 配置并保存 → 版本号递增 (V1.0→V1.1)。
2. 已有论文的 template\_version\_id 仍指向 TemplateVersion(id=3, V1.0), 不受影响。

3. 新论文创建时，系统默认查询当前生效版本（is\_current=true），即使用 V1.1。
4. 若学生希望使用旧版模板，前端模板选择器列出所有历史版本供手动选择。

## 9 Reference 参考文献实体详细设计

### 9.1 数据表结构

参考文献表采用独立表设计（tb\_reference 前缀），不直接绑定论文 ID，支持通用参考文献库管理功能。

表 10: tb\_reference 表结构

| 字段名         | 类型            | 约束                 | 默认值   | 说明                 |
|-------------|---------------|--------------------|-------|--------------------|
| id          | BIGINT        | PK, AUTO_INCREMENT | —     | 主键                 |
| authors     | VARCHAR(500)  | NOT NULL           | —     | 作者（多作者逗号分隔）        |
| title       | VARCHAR(500)  | NOT NULL           | —     | 文献标题               |
| type        | VARCHAR(10)   | NOT NULL           | —     | 文献类型枚举（J/M/C/D/EB） |
| year        | INT           | —                  | —     | 出版年份               |
| journal     | VARCHAR(500)  | —                  | —     | 刊物名称（期刊专用）         |
| volume      | VARCHAR(50)   | —                  | —     | 卷号                 |
| issue       | VARCHAR(50)   | —                  | —     | 期号                 |
| pages       | VARCHAR(50)   | —                  | —     | 页码                 |
| publisher   | VARCHAR(500)  | —                  | —     | 出版社（书籍专用）          |
| address     | VARCHAR(500)  | —                  | —     | 出版地（书籍专用）          |
| conference  | VARCHAR(500)  | —                  | —     | 会议名称（会议专用）         |
| institution | VARCHAR(500)  | —                  | —     | 学位授予单位（学位论文专用）     |
| url         | VARCHAR(1000) | —                  | —     | 访问链接（网络资源专用）       |
| access_date | VARCHAR(100)  | —                  | —     | 引用日期（网络资源专用）       |
| created_at  | TIMESTAMP     | —                  | NOW() | 创建时间               |
| updated_at  | TIMESTAMP     | —                  | NOW() | 更新时间               |

### 9.2 文献类型枚举定义

### 9.3 核心接口规范

参考文献模块提供 RESTful CRUD 接口，统一返回 Result<T> 格式：

Listing 22: ReferenceController 核心接口

```

1 // 获取所有文献（按年份降序）
2 GET    /api/references
3 // Response: Result<List<Reference>>

```

表 11: ReferenceType 枚举值

| 枚举值 | 中文名称  | 特有字段                          |
|-----|-------|-------------------------------|
| J   | 期刊文章  | journal, volume, issue, pages |
| M   | 专著/书籍 | publisher, address            |
| C   | 会议论文  | conference, pages             |
| D   | 学位论文  | institution                   |
| EB  | 网络资源  | url, access_date              |

```

4
5 // 获取单条文献
6 GET      /api/references/{id}
7 // Response: Result<Reference> / 404
8
9 // 新增文献
10 POST    /api/references
11 // RequestBody: Reference JSON
12 // Response: Result<Reference> (201)
13
14 // 更新文献
15 PUT      /api/references/{id}
16 // RequestBody: Reference JSON
17 // Response: Result<Reference> / 404
18
19 // 删除文献（存在性预检）
20 DELETE   /api/references/{id}
21 // Response: Result<Void> / 404
22
23 // 搜索
24 GET      /api/references/search/author?keyword=张三
25 GET      /api/references/search/title?keyword=深度学习
26
27 // 获取格式化文本
28 GET      /api/references/{id}/format
29 // Response: Result<{ formatted: "..."}>
30
31 // 获取全部格式化列表
32 GET      /api/references/format/all
33 // Response: Result<List<String>>

```

## 9.4 GB/T 7714 格式化算法详细设计

格式化器采用策略模式实现，核心方法根据文献类型自动路由到对应的格式化方法。

Listing 23: Gbt7714Formatter 核心算法



```

1 @Component
2 public class Gbt7714Formatter {
3
4     public String format(Reference ref) {
5         if (ref == null) return "";
6         return switch (ref.getType()) {
7             case J -> formatJournal(ref);
8             case M -> formatBook(ref);
9             case C -> formatConference(ref);
10            case D -> formatDissertation(ref);
11            case EB -> formatElectronic(ref);
12        };
13    }
14
15    // [J] 作者. 标题[J]. 刊物, 年份, 卷(期): 页码.
16    private String formatJournal(Reference ref) {
17        StringBuilder sb = new StringBuilder();
18        sb.append(ref.getAuthors()).append(". ");
19        sb.append(ref.getTitle()).append("[J]. ");
20        sb.append(nullToEmpty(ref.getJournal())).append(", ");
21        sb.append(ref.getYear());
22        // 卷、期、页码...
23        sb.append(".");
24        return sb.toString();
25    }
26
27    // [M] 作者. 标题[M]. 出版地: 出版社, 年份.
28    private String formatBook(Reference ref) { /* ... */ }
29
30    // [C] 作者. 标题[C]// 会议名称. 年份: 页码.
31    private String formatConference(Reference ref) { /* ... */ }
32
33    // [D] 作者. 标题[D]. 授予单位, 年份.
34    private String formatDissertation(Reference ref) { /* ... */ }
35
36    // [EB] 作者. 标题[EB/OL]. 链接, 引用日期.
37    private String formatElectronic(Reference ref) { /* ... */ }
38 }

```

关键设计要点:

1. 所有业务字段均做空值保护 (null safe), 避免字段为空时输出字面量“null”;
2. 各格式化方法独立封装, 新增文献类型时仅需添加枚举值和对应私有方法, 不修改现有代码 (开闭原则);
3. 格式化结果不含序号前缀, 由调用方 (导出模块) 在生成列表时统一添加 [1][2] 序号。

## 10 Image 图片实体详细设计

### 10.1 数据表结构

表 12: tb\_image 表结构

| 字段名           | 类型            | 约束                 | 默认值   | 说明              |
|---------------|---------------|--------------------|-------|-----------------|
| id            | BIGINT        | PK, AUTO_INCREMENT | —     | 主键              |
| original_name | VARCHAR(500)  | NOT NULL           | —     | 原始文件名（用户上传时的名字） |
| stored_name   | VARCHAR(500)  | NOT NULL           | —     | 存储文件名（UUID 生成）  |
| file_path     | VARCHAR(1000) | NOT NULL           | —     | 文件在磁盘上的绝对路径     |
| file_size     | BIGINT        | —                  | —     | 文件大小（字节）        |
| content_type  | VARCHAR(100)  | —                  | —     | 文件 MIME 类型      |
| created_at    | TIMESTAMP     | —                  | NOW() | 上传时间            |

### 10.2 图片上传核心流程

Listing 24: ImageService.upload() 核心流程

```

1 public Image upload(MultipartFile file) {
2     // 1. 校验文件非空
3     if (file.isEmpty()) throw new BusinessException(400, "上传文件不能为空");
4
5     // 2. 校验MIME类型（仅允许 jpg/png/gif/webp）
6     String contentType = file.getContentType();
7     if (contentType == null || !ALLOWED_TYPES.contains(contentType))
8         throw new BusinessException(400, "不支持的图片类型");
9
10    // 3. 校验文件大小（最大10MB）
11    if (file.getSize() > MAX_FILE_SIZE)
12        throw new BusinessException(400, "文件大小超出限制");
13
14    // 4. UUID重命名，防止重名
15    String extension = extractExtension(file.getOriginalFilename());
16    String storedName = UUID.randomUUID() + extension;
17    Path targetPath = uploadPath.resolve(storedName);
18
19    // 5. try-with-resources 写入磁盘
20    try (InputStream in = file.getInputStream()) {
21        Files.copy(in, targetPath, REPLACE_EXISTING);
22    }
23
24    // 6. 元信息入库
25    Image image = Image.builder()
26        .originalName(file.getOriginalFilename())

```

```

27         .storedName(storedName)
28         .filePath(targetPath.toString())
29         .fileSize(file.getSize())
30         .contentType(contentType)
31         .build();
32     return imageRepository.save(image);
33 }

```

## 10.3 上传接口规范

**Listing 25:** ImageController 上传与访问接口

```

1  // 上传图片
2  POST /api/images/upload
3  // Content-Type: multipart/form-data
4  // Body: file (MultipartFile)
5  // Response(201): {
6  //     "code": 200,
7  //     "data": {
8  //         "id": 1,
9  //         "originalName": "diagram.png",
10 //         "fileSize": 204800,
11 //         "contentType": "image/png",
12 //         "url": "/api/images/1/file"
13 //     }
14 // }
15
16 // 获取图片元信息
17 GET /api/images/{id}
18
19 // 获取图片文件（浏览器可直接查看）
20 GET /api/images/{id}/file
21 // Response: image/png 流（inline Content-Disposition）
22
23 // 删除图片（删除磁盘文件+数据库记录）
24 DELETE /api/images/{id}

```

## 11 文档导出模块详细设计

### 11.1 总体架构

文档导出模块分为三个核心步骤：

1. **数据准备：**从数据库读取论文完整数据，包括章节 HTML 内容、参考文献格式化列表、图片文件路径、模板格式配置；

2. **DOCX 生成**: 使用 Apache POI XWPF 组件, 按模板格式逐项构建 Word 文档;
3. **PDF 转换**: 调用 LibreOffice 命令行工具, 以无头模式将 DOCX 转换为 PDF。

## 11.2 Apache POI DOCX 生成算法

DOCX 生成的核心流程采用 XWPFDocument 对象逐段构建:

Listing 26: Apache POI DOCX 生成核心逻辑

```

1 public void generateDocx(Thesis thesis, List<ThesisSection> sections,
2                           List<String> formattedRefs,
3                           Map<String, Path> imageMap) throws Exception {
4
5     XWPFDocument document = new XWPFDocument();
6
7     // 1. 生成封面
8     addCover(document, thesis);
9
10    // 2. 遍历章节树, 逐级生成标题和内容
11    for (ThesisSection section : buildTree(sections)) {
12        // 生成章节标题 (按层级设置不同样式)
13        XWPFParagraph titlePara = document.createParagraph();
14        titlePara.setAlignment(ParagraphAlignment.LEFT);
15        XWPFRun titleRun = titlePara.createRun();
16        titleRun.setText(section.getTitle());
17        titleRun.setBold(true);
18        // 一级标题二号(22pt), 二级标题三号(16pt), 正文小四(12pt)
19
20        // 解析HTML内容, 提取纯文本段落
21        String htmlContent = section.getContent();
22        List<String> paragraphs = parseHtmlParagraphs(htmlContent);
23
24        for (String paraText : paragraphs) {
25            XWPFParagraph para = document.createParagraph();
26            para.setIndentationFirstLine(480); // 首行缩进2字符
27            XWPFRun run = para.createRun();
28            run.setText(paraText);
29            run.setFontFamily("宋体");
30            run.setFontSize(12);
31
32            // 解析HTML中的图片<img>标签, 嵌入文档
33            List<String> imgSrcs = extractImageSources(paraText);
34            for (String imgSrc : imgSrcs) {
35                Path imgPath = imageMap.get(imgSrc);
36                if (imgPath != null) {
37                    addImageToDocument(document, imgPath);
38                }
39            }
40        }
41    }
42 }

```

```

40     }
41 }
42
43 // 3. 生成参考文献列表
44 XWPFParagraph refTitle = document.createParagraph();
45 XWPFRun refRun = refTitle.createRun();
46 refRun.setText("参考文献");
47 refRun.setBold(true);
48 refRun.setFontSize(16);
49
50 int index = 1;
51 for (String formatted : formattedRefs) {
52     XWPFParagraph refPara = document.createParagraph();
53     XWPFRun run = refPara.createRun();
54     run.setText "[" + (index++) + "] " + formatted);
55     run.setFontFamily("宋体");
56     run.setFontSize(10.5); // 五号字
57 }
58
59 // 4. 输出到文件
60 try (FileOutputStream out = new FileOutputStream(outputPath)) {
61     document.write(out);
62 }
63 document.close();
64 }

```

### 11.3 LibreOffice PDF 转换

DOCX 文件生成完成后, 通过 Java Runtime 调用 LibreOffice 命令行进行 PDF 转换:

**Listing 27:** LibreOffice PDF 转换命令

```

1 // 转换命令
2 // soffice --headless --convert-to pdf demo.docx
3
4 String libreOfficePath = "C:\\Program Files\\LibreOffice\\program\\soffice.exe";
5 String docxPath = "/path/to/output.docx";
6 String outputDir = "/path/to/output/";
7
8 ProcessBuilder pb = new ProcessBuilder(
9     libreOfficePath,
10    "--headless",
11    "--convert-to", "pdf",
12    "--outdir", outputDir,
13    docxPath
14);
15 pb.redirectErrorStream(true);
16 Process process = pb.start();

```

```
17 int exitCode = process.waitFor();
18
19 if (exitCode == 0) {
20     // 转换成功，在outputDir下生成同名.pdf文件
21 } else {
22     // 转换失败，读取错误流记录日志
23 }
```

关键技术要点：

- **中文支持：**生成 DOCX 时必须显式设置中文字体（宋体），确保 LibreOffice 转换时正确渲染；
- **文件隔离：**每个导出任务在独立临时目录中操作，避免并发写入冲突；
- **超时控制：**设置合理的 Process 执行超时时间（建议 120 秒），超时后强制终止进程防止僵尸进程；
- **异步执行：**导出操作在异步任务线程池中执行，不阻塞用户的其他操作。

11.4 技术验证结论

经技术验证，文档导出方案已达到以下成果：

表 13: 技术验证结果汇总

| 验证项              | 结果 | 说明                           |
|------------------|----|------------------------------|
| POI 创建中文标题       | 通过 | 宋体 22pt，居中加粗，中文无乱码           |
| POI 中文正文段落       | 通过 | 宋体 12pt，首行缩进 2 字符            |
| POI 表格创建         | 通过 | 4×4 表格，单元格内容 + 样式正常          |
| LibreOffice 无头转换 | 通过 | DOCX→PDF 全自动，转换后中文正常         |
| 参考文献格式化          | 通过 | GB/T 7714 五种类型均正确输出          |
| 图片上传与访问          | 通过 | Multipart 上传 → 本地存储 → URL 访问 |

结论：Apache POI 5.x + LibreOffice 无头转换的技术方案可行，可进入正式开发阶段。

12 学生端页面详细设计概述

学生端共涉及 8 个核心页面/对话框，本章逐一描述每个页面的布局结构、控件清单、数据绑定、交互逻辑及输入校验规则。所有页面遵循 Element Plus 设计规范，统一使用 #409EFF（主题蓝）作为主色调，#67C23A（成功绿）、#E6A23C（警告橙）、#F56C6C（危险红）作为辅助功能色。

## 12.1 页面 1：登录页（Login.vue）

### 12.1.1 页面布局

居中双栏卡片布局，整体宽度 960px，垂直水平居中。左侧（占 40% 宽度）为品牌展示区，显示系统 Logo、名称” 论文生成系统”、副标题。右侧（占 60% 宽度）为登录表单区。

### 12.1.2 控件清单

表 14: 登录页控件明细

| 控件类型                    | 字段名      | 控件说明             | 校验规则          |
|-------------------------|----------|------------------|---------------|
| el-input                | username | 学号输入框            | 必填，纯数字，8-12 位 |
| el-input(type=password) | password | 密码输入框，带显示/隐藏图标   | 必填，6 位        |
| el-checkbox             | remember | ” 记住我” 复选框       | 无校验           |
| el-button(type=primary) | —        | ” 登录” 按钮，宽度 100% | 表单校验通过后可用     |
| el-link                 | —        | ” 还没有账号？立即注册” 链接 | —             |

### 12.1.3 交互逻辑

1. 用户输入学号和密码，点击” 登录” 按钮或按 Enter 键提交；
2. 前端调用表单校验，校验通过后调用 `authStore.login(username, password)`；
3. 登录成功后跳转至论文列表页 `/papers`；登录失败在输入框下方显示红色错误提示；
4. 若勾选” 记住我”，Token 存储至 `localStorage`；否则使用 `sessionStorage`。

### 12.1.4 输入校验规则

```
1 const loginRules = {
2   username: [
3     { required: true, message: '请输入学号', trigger: 'blur' },
4     { pattern: /^\\d{8,12}$/, message: '学号为8-12位数字', trigger: 'blur' }
5   ],
6   password: [
7     { required: true, message: '请输入密码', trigger: 'blur' },
8     { min: 6, message: '密码长度不少于6位', trigger: 'blur' }
9   ]
10 }
```

## 12.2 页面 2：注册页（Register.vue）

### 12.2.1 页面布局

布局风格与登录页一致，右侧表单区域扩展为 5 个字段。注册成功后自动跳转至登录页并弹出绿色成功提示。

### 12.2.2 控件清单

表 15: 注册页控件明细

| 控件类型                    | 字段名             | 控件说明   | 校验规则                |
|-------------------------|-----------------|--------|---------------------|
| el-input                | studentId       | 学号     | 必填，数字，8-12 位，后端唯一校验 |
| el-input                | name            | 姓名     | 必填，2-20 个字符         |
| el-input                | email           | 邮箱     | 必填，合法邮箱格式           |
| el-input(type=password) | password        | 密码     | 必填，6 位，含字母 + 数字     |
| el-input(type=password) | confirmPassword | 确认密码   | 必填，与 password 一致    |
| el-button(type=primary) | —               | ”注册”按钮 | 所有校验通过后可用           |

### 12.2.3 交互逻辑

1. 用户填写全部字段，前端实时校验各项输入；
2. 学号输入框在失焦时触发唯一性校验；
3. 全部校验通过后，提交 POST /api/v1/auth/register；
4. 注册成功跳转登录页；注册失败显示具体错误。

### 12.2.4 输入校验规则

```
1 const registerRules = {
2   studentId: [
3     { required: true, message: '请输入学号', trigger: 'blur' },
4     { pattern: /^\\d{8,12}$/, message: '学号为8-12位数字', trigger: 'blur' }
5   ],
6   name: [
7     { required: true, message: '请输入姓名', trigger: 'blur' },
8     { min: 2, max: 20, message: '姓名为2-20个字符', trigger: 'blur' }
9   ],
10  email: [
11    { required: true, message: '请输入邮箱', trigger: 'blur' },
12    { type: 'email', message: '邮箱格式不正确', trigger: 'blur' }
13  ],
14  password: [
```



```
15     { required: true, message: '请输入密码', trigger: 'blur' },
16     { min: 6, message: '密码长度不少于6位', trigger: 'blur' },
17     { pattern: /^(?=.*[a-zA-Z])(?=.*\d)/, message: '密码需包含字母和数字',
      trigger: 'blur' }
18   ],
19   confirmPassword: [
20     { required: true, message: '请确认密码', trigger: 'blur' },
21     { validator: (rule, value, callback) => {
22       if (value !== form.password) callback(new Error('两次密码不一致'))
23       else callback()
24     }, trigger: 'blur' }
25   ]
26 }
```

## 12.3 页面 3：论文列表页（PaperList.vue）

### 12.3.1 页面布局

顶部：el-input 搜索框 + el-select 排序下拉框 + el-button(type=primary)”新建论文”按钮。主体区域：论文卡片网格布局（响应式：1200px 四列，768px 两列，<768px 单列）。

### 12.3.2 控件清单

表 16：论文列表页控件明细

| 控件类型                    | 控件说明               | 数据绑定                    |
|-------------------------|--------------------|-------------------------|
| el-input(搜索)            | 按论文标题搜索，输入防抖 300ms | v-model=”searchKeyword” |
| el-select               | 排序方式选择             | v-model=”sortBy”        |
| el-button(type=primary) | 新建论文按钮             | @click 前往创建页            |
| PaperCard(自定义)          | 论文卡片组件             | v-for 循环渲染              |
| el-pagination           | 分页器（每页 12 条）       | v-model:current-page    |
| el-empty                | 空状态插画              | v-if 空列表                |

### 12.3.3 论文卡片子组件设计

每张论文卡片（PaperCard）包含：

- 论文标题（el-text，最多显示 2 行，超出省略号截断）；
- 模板名称标签（el-tag，type=info，size=small）；
- 最后修改时间（相对时间格式：“3 小时前”、“昨天 14:30”）；
- 字数统计（”约 12,500 字”）；

- 操作按钮组：编辑（type=primary, link）、预览（type=success, link）、删除（type=danger, link）。

#### 12.3.4 交互逻辑

1. 页面挂载时调用 GET /api/v1/papers 获取论文列表；
2. 搜索关键词变更（防抖 300ms）或排序方式变更时，重新请求后端接口；
3. 点击”编辑”按钮，跳转至 /editor/:paperId；
4. 点击”预览”按钮，在新标签页打开 /preview/:paperId；
5. 点击”删除”按钮，弹出 ElMessageBox 二次确认对话框，确认后删除；
6. 论文卡片 hover 时浮现浅色阴影和上移动画。

### 12.4 页面 4：论文创建向导（PaperCreate.vue）

#### 12.4.1 页面布局

使用 el-steps 步骤条引导三步创建流程：基本信息 → 选择模板 → 确认创建。

#### 12.4.2 控件清单

表 17: 论文创建向导控件明细

| 步骤  | 控件类型                    | 控件说明         | 校验规则        |
|-----|-------------------------|--------------|-------------|
| 第一步 | el-input                | 论文标题         | 必填，1-200 字符 |
|     | el-select               | 所属学院选择       | 必选          |
| 第二步 | TemplateCard(自定义)       | 模板卡片列表，单选    | 必选          |
| 第三步 | el-descriptions         | 汇总展示标题、学院、模板 | —           |
|     | el-button(type=primary) | ”确认创建”按钮     | —           |

#### 12.4.3 交互逻辑

1. 步骤条已完成步骤可点击回退，未完成步骤不可跳过；
2. 第一步选择学院后，第二步自动加载对应模板列表；
3. 第二步点击模板卡片进行单选，高亮选中状态；
4. 第三步展示汇总信息，点击”确认创建”提交，成功后自动跳转编辑器。

## 12.5 页面 5：论文编辑器主页面（PaperEditor.vue）

这是学生端最核心、最复杂的页面，采用三栏可调整布局。

### 12.5.1 页面布局

整体高度 100vh，三栏布局：

- 顶部工具栏（高度 48px）：论文标题显示 + 保存状态指示器 + 预览按钮 + 导出按钮 + 返回列表按钮；
- 左侧章节树面板（默认 260px，可拖拽宽度）：章节树控件 + 添加章节按钮，可收起至 40px；
- 中心编辑区（flex:1）：顶部格式工具栏 + Tiptap 富文本编辑区域；
- 右侧辅助面板（默认 320px）：Tab 切换”参考文献” / ”模板信息”，可收起。

### 12.5.2 控件清单

顶部工具栏：

- el-text(tag="h3") — 论文标题，点击可编辑
- SaveIndicator(自定义) — 保存状态：”已保存 15:30” / ”保存中...” / ”未保存 ”
- el-button(plain) — ”预览”按钮，新标签页打开预览
- el-button(type=primary) — ”导出”按钮，弹出 ExportDialog
- el-button(text) — ”← 返回列表”按钮，含未保存确认

左侧章节树面板：

- el-tree — 章节树，支持 node 拖拽排序，自定义节点渲染，右键上下文菜单
- el-button(text) — ”+ 添加章节”按钮

章节树右键菜单（自定义 el-dropdown）：

- ”添加同级章节”、”添加子章节”、”重命名”（章节名称变为 el-input）、”删除章节”（含子章节提示）

中心编辑区格式工具栏：

右侧辅助面板（Tab 切换）：

- 参考文献管理 Tab：el-table(列表) + el-button(添加) + 行内编辑/删除/引用按钮
- 模板信息 Tab：当前模板名称 + 样式描述 + 缩略图 + el-button(”切换模板”)

表 18: 编辑器格式工具栏控件

| 分组    | 控件                             | Tiptap 命令                          |
|-------|--------------------------------|------------------------------------|
| 文本样式  | el-button-group(加粗/斜体/下划线/删除线) | toggleBold/Italic/Underline/Strike |
| 标题层级  | el-select(正文/H1/H2/H3/H4)      | toggleHeading                      |
| 文字颜色  | el-color-picker(文字色/背景色)       | setColor/setHighlight              |
| 对齐方式  | el-button-group(左/中/右/两端)      | setTextAlign                       |
| 列表    | el-button-group(有序/无序)         | toggleOrderedList/BulletList       |
| 缩进    | el-button-group(增/减缩进)         | indent/outdent                     |
| 插入    | el-button(引用/图片/表格)            | 打开对应面板                             |
| 撤销/重做 | el-button-group                | undo/redo                          |

### 12.5.3 编辑器核心交互流程

1. 页面初始化: 获取 paperId → 并行请求论文详情和章节树 → 创建 Tiptap 实例 → 加载首章节 → 注册 beforeunload 监听
2. 章节切换: 点击 el-tree 节点 → 检查未保存修改 → 执行保存 → 请求新章节内容 → editor.commands.setContent()
3. 自动保存 (useAutoSave composable): 监听 editor onUpdate → 标记脏数据 → 30 秒防抖 → 自动 PUT 请求 → 更新保存状态
4. 章节拖拽: el-tree draggable → node-drop 事件 → PUT 提交新顺序 → 失败则回滚

### 12.5.4 章节名称校验规则

```

1 const sectionNameRules = [
2   { required: true, message: '请输入章节名称', trigger: 'blur' },
3   { max: 100, message: '章节名称不超过100个字符', trigger: 'blur' },
4   { pattern: /^[^\<\>:\\"/\\|\?*\]+$/, message: '不能包含特殊字符', trigger: 'blur' }
5 ]

```

## 12.6 页面 6: 参考文献管理面板 (ReferencePanel.vue)

### 12.6.1 布局

侧边面板内垂直布局: 顶部”添加文献”按钮, 主体 el-table (列: 序号、作者、标题、年份、操作), 底部统计”共 N 条文献”。

### 12.6.2 参考文献表单校验规则

```
1 const referenceRules = {
2   type: [{ required: true, message: '请选择文献类型', trigger: 'change' }],
3   author: [{ required: true, message: '请输入作者', trigger: 'blur' }],
4   title: [{ required: true, message: '请输入标题', trigger: 'blur' },
5     { max: 500, message: '标题不超过500字符', trigger: 'blur' }],
6   journal: [{ required: true, message: '请输入期刊/出版社', trigger: 'blur' }],
7   year: [{ required: true, message: '请输入年份', trigger: 'blur' },
8     { pattern: /^\\d{4}$/, message: '年份为4位数字', trigger: 'blur' }],
9   doi: [{ pattern: /^10\\.\\d+/, message: 'DOI格式不正确', trigger: 'blur' }]
10 }
```

### 12.6.3 交互逻辑

1. 面板展开时加载已有文献列表；
2. 添加/编辑操作弹出 el-dialog，表单字段根据文献类型动态显示；
3. 删除操作弹出确认框，若正文有引用则提示“正文中有 N 处引用该文献”；
4. 用户点击文献行“引用”按钮，编辑器光标位置插入引用标记节点。

## 12.7 页面 7：导出对话框（ExportDialog.vue）

### 12.7.1 布局

el-dialog 模态对话框（宽度 560px），纵向排列导出配置选项。

### 12.7.2 控件清单

表 19: 导出对话框控件明细

| 控件类型                    | 控件说明                         | 默认值  |
|-------------------------|------------------------------|------|
| el-radio-group          | 导出格式（DOCX / PDF）             | PDF  |
| el-checkbox-group       | 导出范围（全部/自定义）+ el-tree-select | 全部章节 |
| el-checkbox             | ”包含封面页”                      | 勾选   |
| el-checkbox             | ”包含目录”                       | 勾选   |
| el-checkbox             | ”包含参考文献列表”                   | 勾选   |
| el-progress             | 导出进度条（提交后显示）                 | 0%   |
| el-button(type=primary) | ”开始导出” / ”下载文件”              | 状态切换 |

### 12.7.3 交互逻辑

1. 选择格式和范围 → 提交 POST /api/v1/papers/:id/export → 返回 taskId;

2. 前端每 2 秒轮询 GET /api/v1/export/:taskId/status → 更新进度条;
3. 完成时按钮变为” 下载文件”, 触发浏览器下载;
4. 失败时显示红色错误信息 + ” 重新导出” 按钮。

## 12.8 页面 8: 个人中心页 (Profile.vue)

### 12.8.1 布局

el-tabs 切换三个子面板: 个人信息、修改密码、导出历史。el-form 表单布局 (label-width=”100px”)。

### 12.8.2 控件摘要

- 个人信息 Tab: el-input (姓名、邮箱、手机号) + el-button(” 保存修改”)
- 修改密码 Tab: el-input(type=password) (原密码、新密码、确认新密码) + el-button(” 修改密码”)
- 导出历史 Tab: el-table (文件名/格式/时间/状态/下载按钮)

### 12.8.3 校验规则

```
1 const profileRules = {
2   name: [{ required: true, min: 2, max: 20, message: '姓名为2-20个字符', trigger:
3     'blur' }],
4   email: [{ required: true, type: 'email', message: '邮箱格式不正确', trigger: '
5     blur' }],
6   phone: [{ pattern: /^1[3-9]\d{9}$/, message: '手机号格式不正确', trigger: 'blur
7     ' }]
8 }
9
10 const passwordRules = {
11   oldPassword: [{ required: true, message: '请输入原密码', trigger: 'blur' }],
12   newPassword: [
13     { required: true, min: 6, message: '密码长度不少于6位', trigger: 'blur' },
14     { pattern: /^(?=.*[a-zA-Z])(?=.*\d)/, message: '密码需包含字母和数字',
15       trigger: 'blur' }
16   ],
17   confirmPassword: [
18     { required: true, message: '请确认新密码', trigger: 'blur' },
19     { validator: (rule, value, callback) => {
20       if (value !== form.newPassword) callback(new Error('两次密码不一致'))
21       else callback()
22     }, trigger: 'blur' }
23   ]
24 }
```

## 13 全局交互规范

### 13.1 消息提示规范

- 操作成功: `ElMessage.success()`，绿色，显示 2 秒；
- 操作失败: `ElMessage.error()`，红色，显示 3 秒，附带具体错误原因；
- 删除确认: 统一使用 `ElMessageBox.confirm()`。

### 13.2 加载状态规范

- 页面级加载: `el-skeleton` 骨架屏；
- 按钮级加载: `el-button` 的 `:loading` 属性；
- 局部刷新: `el-table` 的 `v-loading` 指令。

### 13.3 响应式适配断点

- 桌面端 (1280px): 论文列表 4 列，编辑器三栏完整；
- 平板端 (768-1279px): 论文列表 2 列，编辑器侧面板默认收起；
- 移动端 (<768px): 论文列表 1 列，编辑器仅编辑区 + 抽屉式章节列表。

## 14 管理员端页面详细设计

### 14.1 页面 1: 仪表盘页 (AdminDashboard.vue)

#### 14.1.1 页面布局

仪表盘页采用四卡片 + 双图表的网格布局，上方为四个指标卡片排列成一行，下方为两个图表并排展示。页面顶部右侧提供时间范围选择器 (`el-date-picker`)，筛选后全局刷新图表数据。

#### 14.1.2 控件清单

表 20: 仪表盘页控件明细

| 控件类型            | 用途       | 数据来源                                  | 交互逻辑       |
|-----------------|----------|---------------------------------------|------------|
| StatCard (自定义)  | 显示统计数字   | GET /api/admin/stats                  | 页面加载时一次性获取 |
| el-date-picker  | 时间范围筛选   | —                                     | 选择触发图表数据刷新 |
| Chart (ECharts) | 论文状态分布饼图 | GET /api/admin/stats/papers-by-status | 按选中时间范围查询  |
| Chart (ECharts) | 导出趋势折线图  | GET /api/admin/stats/export-trend     | 按选中时间范围查询  |

14.1.3 数据接口

Listing 28: 仪表盘数据接口定义

```
1 // 获取统计数据概览
2 GET /api/admin/stats
3 Response: {
4   totalUsers: 150,      // 总用户数
5   totalStudents: 120,   // 学生数
6   totalTeachers: 25,    // 教师数
7   totalAdmins: 5,       // 管理员数
8   totalPapers: 200,     // 论文总数
9   totalTemplates: 10,   // 模板总数
10  todayExports: 15      // 今日导出数
11 }
12
13 // 获取论文状态分布
14 GET /api/admin/stats/papers-by-status?start=&end=
15 Response: [{ status: "DRAFT", count: 80 }, ...]
16
17 // 获取导出趋势
18 GET /api/admin/stats/export-trend?start=&end=
19 Response: [{ date: "2026-07-10", count: 5 }, ...]
```

14.2 页面 2：学院管理页（CollegeAdmin.vue）

14.2.1 控件清单

表 21: 学院管理页控件明细

| 控件类型                | 字段/用途   | 校验规则      | 数据操作         |
|---------------------|---------|-----------|--------------|
| el-table            | 学院列表展示  | —         | 行数据由 API 填充  |
| el-input            | 搜索框     | 最大长度 50   | 输入触发前端过滤     |
| el-dialog + el-form | 新增/编辑表单 | 名称非空、代码唯一 | POST/PUT API |
| el-button           | 删除操作    | —         | 二次确认后 DELETE |

14.2.2 交互逻辑

- 1. 页面加载时调用 GET /api/admin/colleges 获取学院列表，渲染至 el-table;
- 2. 点击”新增学院”按钮，弹出 Dialog，表单包含 name（必填）、code（必填，唯一）两字段;
- 3. 表单校验通过后提交 POST /api/admin/colleges，成功后关闭 Dialog 并刷新列表;
- 4. 点击编辑图标，弹出 Dialog 预填当前行数据，提交 PUT /api/admin/colleges/id;



5. 点击删除图标，弹出二次确认框，确认后调用 `DELETE /api/admin/colleges/id`，若服务端返回关联校验错误（如存在关联模板），在前端以 `ElMessage.error` 展示错误信息。

## 14.3 页面 3：模板管理页（TemplateAdmin.vue）

### 14.3.1 页面布局

双栏布局：左侧为模板卡片列表（宽度 320px），右侧为模板编辑面板（自适应宽度）。左侧上方提供筛选栏（按类型下拉选择 + 按学院下拉选择 + 搜索框）。右侧编辑面板使用 Tabs 组织三个配置子页面。

### 14.3.2 模板卡片组件设计

每个模板卡片展示：模板名称（加粗 16px）、类型标签（el-tag，颜色按类型区分：毕业 = 蓝色、课程 = 绿色、项目 = 橙色）、当前版本号、启用状态开关（el-switch，独立提交 `PUT /api/admin/templates/id/toggle`）。卡片选中状态以左侧蓝色边框高亮标记。

### 14.3.3 封面字段配置面板

封面字段配置采用拖拽排序的列表实现：

- 列表项显示字段标签（如“论文题目”）、是否必填标记、字段类型（文本/日期/下拉选择）；
- 每个列表项右侧提供编辑和删除按钮；
- 列表底部有“添加字段”按钮，点击弹出字段配置 Dialog（字段标签、是否必填、字段类型、默认值）；
- 拖拽使用 `vuedraggable` 库实现，拖拽完成后自动更新 `cover_fields`

### 14.3.4 章节结构配置面板

使用 `el-tree` 组件展示树形章节结构，每个节点右侧显示章节类型标识（chapter/section/subsection）。根节点（论文标题）不可删除。交互功能包括：

- 右键菜单（@node-contextmenu 事件）：新增同级章节、新增子章节、重命名、删除、上移、下移；
- 拖拽（draggable 属性）：跨层级拖拽调整章节归属关系，拖拽时校验不产生循环引用；
- 节点点击选中后，右侧展示该章节的详细配置（章节标题输入框、章节类型下拉选择、默认内容占位符）。

14.3.5 格式参数配置面板

格式参数配置采用 el-form 垂直布局，分组排列：

- 页面设置组：纸张大小下拉选择（A4/Legal/Letter）、上/下/左/右边距数字输入框（el-input-number，单位 cm，步进 0.1）；
- 字体设置组：正文字体下拉选择、正文字号下拉选择、一级标题字体/字号、二级标题字体/字号、三级标题字体/字号；
- 段落设置组：行距倍数滑块（el-slider，范围 1.0-2.5，步进 0.1）、段前间距数字输入框、段后间距数字输入框；
- 页眉页脚组：奇偶页眉内容输入框、页码格式下拉选择、页码位置单选组。

配置变更时，顶部的”保存”按钮由禁用状态变为可用，底部显示”有未保存的更改”提示条。保存时调用 PUT /api/admin/templates/id/config，后端自动执行版本递增逻辑。

15 教师端页面详细设计

15.1 页面 1：批阅任务列表页（ReviewList.vue）

15.1.1 页面布局

顶部为筛选操作栏，主体为表格视图。筛选栏包含：学院下拉选择框、论文类型下拉选择框、状态选项卡（全部/待批阅/已批阅/已退回）。

15.1.2 控件清单

表 22: 批阅任务列表页控件明细

| 控件类型      | 用途     | 数据来源                     | 交互                           |
|-----------|--------|--------------------------|------------------------------|
| el-tabs   | 状态筛选   | —                        | 切换触发数据重新加载                   |
| el-select | 学院筛选   | GET /api/colleges        | 联动刷新列表                       |
| el-table  | 论文列表展示 | GET /api/teacher/reviews | 分页加载                         |
| el-tag    | 状态标签   | 状态字段映射                   | 颜色区分：橙 = 待批阅、绿 = 已批阅、灰 = 已退回 |
| el-button | 进入批阅   | —                        | 点击跳转 /teacher/reviews/id     |

15.1.3 数据接口

Listing 29: 批阅列表接口定义

```
1 // 获取批阅任务列表
2 GET /api/teacher/reviews?status=&collegeId=&page=&size=
```

```
3 Response: {
4   records: [{
5     id: 1,
6     thesisId: 10,
7     thesisTitle: "基于Spring Boot的在线论文生成系统",
8     studentName: "张三",
9     collegeName: "计算机科学与技术学院",
10    submittedAt: "2026-07-14 10:30:00",
11    status: "SUBMITTED",    // SUBMITTED / REVIEWED / RETURNED
12    annotationCount: 0,    // 批注数量
13    score: null,           // 分数
14    grade: null            // 等级
15  }],
16  total: 50,
17  page: 1,
18  size: 20
19 }
```

## 15.2 页面 2：论文批阅页（ReviewDetail.vue）

### 15.2.1 页面布局

三栏布局。左侧栏宽度 260px 展示章节树导航，中间栏自适应宽度展示论文全文预览，右侧栏宽度 340px 展示批注管理面板。顶部操作栏包含论文标题、学生信息、操作按钮区（评阅完成按钮、返回列表按钮）。

### 15.2.2 章节树面板

使用 el-tree 组件展示论文完整章节树，章节节点显示标题和该章节的批注数量角标。当前选中的章节高亮显示。教师点击章节节点时切换中间栏的预览内容，同时右侧批注面板加载对应章节的批注列表。

### 15.2.3 论文预览区

论文内容以 HTML 渲染展示，容器样式模拟 A4 纸张效果（白色背景、阴影边框、标准页边距）。核心交互逻辑：

- 使用 contenteditable=false 的 div 容器展示 HTML 内容，确保教师不可编辑；
- 已批注的文字以黄色背景色（#FFF5CC）高亮标记，并添加波浪下划线样式；
- 教师选中文字时触发 selectionchange 事件，获取选中文本的起始偏移量和长度；
- 选中文字后在选中区域上方浮动出现”添加批注（Alt+A）”按钮，点击后触发右侧批注编辑区。

### 15.2.4 批注定位算法

教师选中文字时，需要精确记录选中文字在原始 HTML 内容中的位置，以便将来再次打开批阅时能正确定位到同一段文字。

**Listing 30:** 批注位置定位算法

```

1 // 获取选中文字在HTML纯文本中的偏移量
2 function getSelectionOffset(container) {
3     const selection = window.getSelection();
4     if (!selection.rangeCount) return null;
5
6     const range = selection.getRangeAt(0);
7     const preRange = document.createRange();
8     preRange.selectNodeContents(container);
9     preRange.setEnd(range.startContainer, range.startOffset);
10
11     // 计算纯文本起始偏移（忽略HTML标签）
12     const startOffset = preRange.toString().length;
13     const textLength = selection.toString().length;
14
15     return { startOffset, textLength, selectedText: selection.toString() };
16 }
17
18 // 打开批阅时定位到已有批注位置
19 function scrollToAnnotation(annotation) {
20     const container = document.getElementById('preview-content');
21     const text = container.textContent || container.innerText;
22     const beforeText = text.substring(0, annotation.startOffset);
23     const targetText = text.substring(
24         annotation.startOffset,
25         annotation.startOffset + annotation.textLength
26     );
27
28     // 使用 Range API 定位并高亮
29     const range = document.createRange();
30     // 遍历文本节点查找目标位置（省略具体遍历实现）
31     // 定位后滚动到可见区域
32     range.collapse(true);
33     const rect = range.getBoundingClientRect();
34     container.scrollTop += rect.top - 200; // 偏移200px保证上下文可见
35 }

```

### 15.2.5 批注面板

右侧面板分为上下两部分：

- 上部一批注列表：按创建时间降序排列当前章节的全部批注。每条批注展示：批注内容摘要

（截取前 50 字）、被选中原文（灰色斜体，用引号包裹）、创建时间（相对时间如”3 分钟前”）。点击批注项时，中间栏自动滚动定位到对应原文位置；

- 下部一批注编辑区：使用 textarea 组件展示教师输入的批注内容。当教师选中原文时，自动填充”选中文字”预览区域（只读），textarea 获取焦点等待输入。提供”提交批注”按钮（POST /api/annotations）和”取消”按钮。

15.3 评分对话框详细设计

15.3.1 控件布局

评分对话框使用 el-dialog 组件，宽度 620px，标题为”评阅论文”。内容表单分三部分：

1. 评语区域：使用 Tiptap 轻量级富文本编辑器（或 el-input type=”textarea”），placeholder 为”请输入整体评语（可选）”，最大长度 5000 字符；
2. 评分区域：el-slider 水平滑块，最小值 0，最大值 100，步进 1，显示当前分值（大号数字：48px 字体），评分下方自动展示对应等级标签（el-tag）：
  - ≥90：红色标签”优秀”
  - 80-89：蓝色标签”良好”
  - 70-79：黄色标签”中等”
  - 60-69：绿色标签”及格”
  - <60：灰色标签”不及格”
3. 操作区域：el-radio-group 横向排列，”通过（REVIEWED）”为绿色按钮样式，”退回（RETURNED）”为红色按钮样式。选择”退回”后下方动态展开 textarea 输入框用于填写退回原因（必填，最小 10 字符）。

15.3.2 校验规则

表 23: 评分对话框校验规则

| 字段   | 规则                          | 错误提示            | 触发时机 |
|------|-----------------------------|-----------------|------|
| 评语   | 最大 5000 字符                  | 超出长度限制          | 输入时  |
| 评分   | 0-100 整数                    | 请输入 0-100 之间的整数 | 提交时  |
| 操作类型 | 必选                          | 请选择操作类型         | 提交时  |
| 退回原因 | 操作类型 =RETURNED 时必填，最小 10 字符 | 退回原因至少 10 个字符   | 提交时  |

### 15.3.3 提交流程

1. 教师点击 Dialog 的” 确认提交” 按钮；
2. 前端执行校验：评分范围、退回原因条件必填；
3. 校验通过后调用 POST /api/teacher/reviews，请求体包含 thesisId、commentHtml、score、action、returnReason（可选）；
4. 提交成功后关闭 Dialog，前端显示 ElMessage.success(” 评阅完成”)，跳转回批阅任务列表页；
5. 提交失败时显示 ElMessage.error，保留当前 Dialog 中的表单数据供教师修正。

## 15.4 前端状态管理

教师端批阅页面涉及多个组件间的状态共享，使用 Pinia store 进行管理：

**Listing 31:** 批阅页面 Pinia Store 定义

```

1 // stores/review.ts
2 export const useReviewStore = defineStore('review', () => {
3   // 当前批阅的论文信息
4   const currentThesis = ref(null)
5   // 章节树列表
6   const sections = ref([])
7   // 当前选中的章节ID
8   const activeSectionId = ref(null)
9   // 当前章节的全部批注
10  const annotations = ref([])
11  // 当前选中的文本信息（用于添加批注）
12  const selectedText = ref(null)
13  // 加载状态
14  const loading = ref(false)
15
16  // 加载论文批阅数据
17  async function loadReview(thesisId) {
18    loading.value = true
19    const [thesisRes, sectionsRes] = await Promise.all([
20      api.getThesisDetail(thesisId),
21      api.getSections(thesisId)
22    ])
23    currentThesis.value = thesisRes.data
24    sections.value = sectionsRes.data
25    loading.value = false
26  }
27
28  // 切换章节时更新批注列表
29  async function switchSection(sectionId) {
30    activeSectionId.value = sectionId

```

```
31     const res = await api.getAnnotations(currentThesis.value.id, sectionId)
32     annotations.value = res.data
33   }
34
35   return {
36     currentThesis, sections, activeSectionId,
37     annotations, selectedText, loading,
38     loadReview, switchSection
39   }
40 })
```

## 16 批阅模块、缓存策略与部署详细设计

本章对批阅子系统、Redis缓存策略以及容器化部署方案进行详细设计。批阅子系统涵盖论文提交、教师批注、评语打分与退回重改四大核心流程；Redis 策略部分给出缓存 Key 规范、序列化方案及防穿透/击穿/雪崩的工程实现；部署部分提供完整的 Dockerfile、dockercompose 编排文件与环境配置模板。

### 16.1 批阅模块详细设计

批阅模块是连接学生与教师的核心交互子系统。学生在完成论文编辑后提交论文进入批阅流程；教师对论文各章节添加批注，最终给出综合评语与打分（通过或退回）。本节从数据模型、服务层接口、RESTful API 与流程设计四方面展开。

#### 16.1.1 数据模型

模块涉及三张核心数据表：`submission`（提交记录）、`annotation`（教师批注）和 `review_record`（评语/批阅记录），以及关联的 `thesis`（论文）与 `thesis_section`（章节）。

##### （1）Submission —— 提交记录

表 24: Submission 表字段说明

| 字段             | 类型        | 约束                  | 说明           |
|----------------|-----------|---------------------|--------------|
| id             | BIGSERIAL | PK                  | 主键自增         |
| thesis_id      | BIGINT    | FK NOT NULL         | 关联论文         |
| student_id     | BIGINT    | FK NOT NULL         | 提交学生         |
| version_number | INT       | NOT NULL, DEFAULT 1 | 提交版本号，每次提交递增 |
| submitted_at   | TIMESTAMP | DEFAULT NOW         | 提交时间         |

##### （2）Annotation —— 教师批注

批注采用定位式设计：前端富文本编辑器在用户选中某段文字并添加批注时，记录选中文字相对于章节正文的 `start_offset` 与 `text_length`。重新渲染时根据偏移量定位到原文位置并高亮显示批注标记。

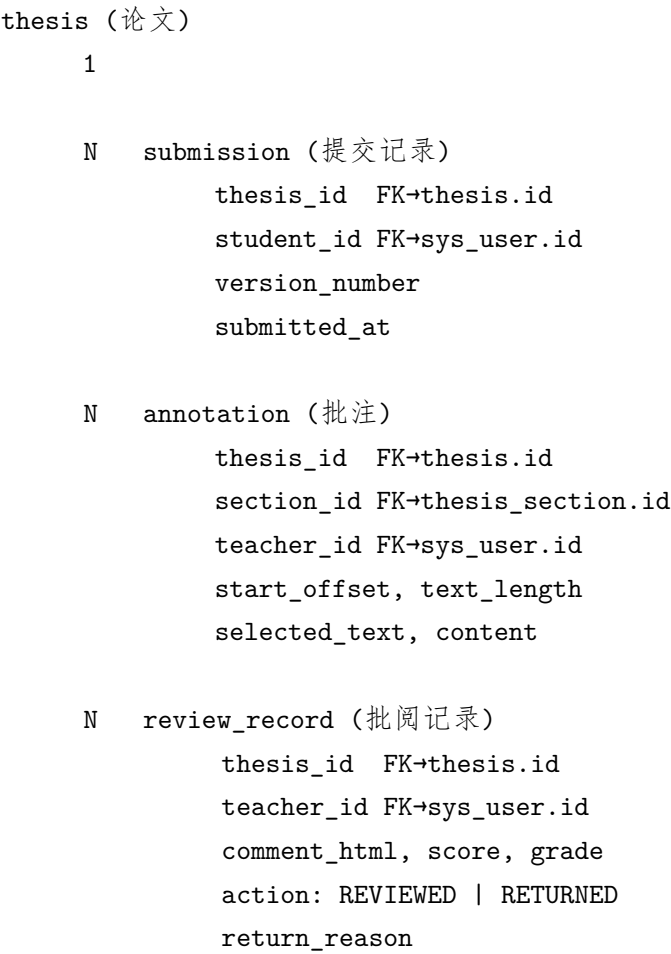


图 2: 批阅模块实体关系图

表 25: Annotation 表字段说明

| 字段            | 类型        | 约束          | 说明              |
|---------------|-----------|-------------|-----------------|
| id            | BIGSERIAL | PK          | 主键自增            |
| thesis_id     | BIGINT    | FK NOT NULL | 关联论文            |
| section_id    | BIGINT    | FK NOT NULL | 关联具体章节          |
| teacher_id    | BIGINT    | FK NOT NULL | 批注教师            |
| start_offset  | INT       | NOT NULL    | 选中文本在章节内容中的起始偏移 |
| text_length   | INT       | NOT NULL    | 选中文本长度          |
| selected_text | TEXT      | 可空          | 被标注的原文内容        |
| content       | TEXT      | NOT NULL    | 教师的批注意见         |
| created_at    | TIMESTAMP | DEFAULT NOW | 创建时间            |



### (3) ReviewRecord ——批阅记录

表 26: ReviewRecord 表字段说明

| 字段            | 类型          | 约束            | 说明                         |
|---------------|-------------|---------------|----------------------------|
| id            | BIGSERIAL   | PK            | 主键自增                       |
| thesis_id     | BIGINT      | FK NOT NULL   | 关联论文                       |
| teacher_id    | BIGINT      | FK NOT NULL   | 批阅教师                       |
| comment_html  | TEXT        | 可空            | 富文本综合评语                    |
| score         | INT         | CHECK [0,100] | 评分（0-100 分制）               |
| grade         | VARCHAR(10) | 可空            | 等级映射（优/良/中/及格/不及格）         |
| action        | VARCHAR(20) | NOT NULL      | REVIEWED（通过）或 RETURNED（退回） |
| return_reason | TEXT        | 可空            | 退回原因（action=RETURNED 时必填）  |
| created_at    | TIMESTAMP   | DEFAULT NOW   | 创建时间                       |

#### 16.1.2 服务层设计

为三层实体分别定义 Service 接口与实现类，统一继承项目已有的分层架构规范。

**SubmissionService 核心方法：**

Listing 32: SubmissionService 接口核心方法

```

1 public interface SubmissionService {
2     /**
3      * 学生提交论文：校验论文归属、更新状态为 SUBMITTED、
4      * 生成版本号递增的提交记录
5      */
6     Submission submitThesis(Long thesisId, Long studentId);
7
8     /** 查询某论文的全部提交历史（按时间倒序） */
9     List<Submission> getSubmissionHistory(Long thesisId);
10 }

```

提交逻辑实现要点：

1. 校验 thesisId 对应的论文状态是否为 DRAFT 或 RETURNED（只有这两种状态允许提交）。
2. 校验 studentId 是否为该论文的所有者（防止越权提交他人论文）。
3. 查询当前最大 version\_number，新提交的版本号 = max + 1。
4. 创建 Submission 记录，更新 thesis.status = SUBMITTED。
5. 整个操作包裹在 @Transactional 中保证原子性。

**AnnotationService 核心方法：**

Listing 33: AnnotationService 接口核心方法

```

1 public interface AnnotationService {
2     /** 教师创建批注 */
3     Annotation createAnnotation(Long thesisId, Long sectionId,
4         Long teacherId, int startOffset, int textLength,
5         String selectedText, String content);
6
7     /** 教师修改批注内容 */
8     Annotation updateAnnotation(Long annotationId, String newContent);
9
10    /** 删除单个批注 */
11    void deleteAnnotation(Long annotationId);
12
13    /** 查询某论文全部批注（按创建时间排序） */
14    List<Annotation> getAnnotationsByThesis(Long thesisId);
15
16    /** 查询某论文某章节的全部批注（按偏移量排序） */
17    List<Annotation> getAnnotationsBySection(Long thesisId, Long sectionId);
18 }

```

批注创建实现要点：

1. 校验 teacherId 对应角色的用户是否为 TEACHER。
2. 校验 sectionId 对应的章节是否属于 thesisId 对应的论文（防止跨论文越权批注）。
3. start\_offset 与 text\_length 合法性校验： $\text{offset} \geq 0$ ， $\text{length} > 0$ ， $\text{offset} + \text{length} \leq \text{章节正文总长度}$ 。
4. 创建 Annotation 记录后返回给前端，前端根据偏移量在编辑器中渲染高亮标记。

ReviewRecordService 核心方法：

Listing 34: ReviewRecordService 接口核心方法

```

1 public interface ReviewRecordService {
2     /** 教师提交批阅结果 */
3     ReviewRecord submitReview(Long thesisId, Long teacherId,
4         String commentHtml, Integer score, String grade,
5         String action, String returnReason);
6
7     /** 查询某论文的全部批阅记录 */
8     List<ReviewRecord> getReviewHistory(Long thesisId);
9 }

```

批阅逻辑实现要点：

1. 校验论文状态必须为 SUBMITTED（已提交但尚未批阅的论文才可批阅）。
2. 若 action = RETURNED, returnReason 不可为空，此时将 thesis.status 更新为 RETURNED。



- DRAFT → COMPLETED: 学生确认论文编辑完成（前端触发，非提交动作）。
- COMPLETED → SUBMITTED: 学生调用提交接口，创建 Submission 记录。
- SUBMITTED → REVIEWED: 教师打回通过结果（action=REVIEWED），论文定稿。
- SUBMITTED → RETURNED: 教师打回退回结果（action=RETURNED），须填写退回原因，学生可查看后修改并重新提交。
- RETURNED → SUBMITTED: 学生修改后重新提交（视为下一版本）。

## 16.2 Redis 缓存策略详细设计

本节给出 Redis 在业务层中的具体应用模式与 Java 代码实现范例，涵盖缓存读取、写入、失效与防穿透/击穿/雪崩的工程手段。

### 16.2.1 Cache-Aside 模式实现

以学院列表缓存为例，展示标准的 Cache-Aside 读写流程：

**Listing 35:** 学院列表缓存的 Cache-Aside 实现

```

1 @Service
2 @RequiredArgsConstructor
3 public class CollegeService {
4     private final CollegeRepository collegeRepo;
5     private final RedisTemplate<String, Object> redisTemplate;
6     private static final String CACHE_KEY = "college:list";
7     private static final Duration TTL = Duration.ofHours(1);
8
9     public List<College> getAllColleges() {
10         // 1. 查缓存
11         Object cached = redisTemplate.opsForValue().get(CACHE_KEY);
12         if (cached != null) {
13             return (List<College>) cached;
14         }
15         // 2. 缓存未命中，查数据库
16         List<College> colleges = collegeRepo.findAll();
17         // 3. 写入缓存（带 TTL + 随机偏移防雪崩）
18         if (!colleges.isEmpty()) {
19             Duration ttl = TTL.plusMillis(
20                 ThreadLocalRandom.current().nextLong(0, 600_000)); // ±10min
21             redisTemplate.opsForValue().set(CACHE_KEY, colleges, ttl);
22         } else {
23             // 4. 缓存空值防穿透，短 TTL
24             redisTemplate.opsForValue().set(CACHE_KEY, "NULL",
25                 Duration.ofSeconds(60));
26         }
27         return colleges;

```

```

28     }
29
30     public College saveCollege(College college) {
31         College saved = collegeRepo.save(college);
32         // 5. 数据变更时主动删除缓存
33         redisTemplate.delete(CACHE_KEY);
34         return saved;
35     }
36 }

```

### 16.2.2 章节草稿自动保存

学生编辑论文时，前端每隔 30 秒将当前章节的草稿内容通过 API 暂存至 Redis。Redis 的高写入性能天然适合此类高频小型写入场景，避免对 PostgreSQL 的频繁更新。

**Listing 36:** 草稿自动保存实现

```

1  @Service
2  @RequiredArgsConstructor
3  public class DraftService {
4      private final RedisTemplate<String, Object> redisTemplate;
5      private static final Duration DRAFT_TTL = Duration.ofMinutes(30);
6
7      public void saveDraft(Long thesisId, Long sectionId,
8                          String draftContent) {
9          String key = "draft:" + thesisId + ":" + sectionId;
10         redisTemplate.opsForValue().set(key, draftContent, DRAFT_TTL);
11     }
12
13     public String getDraft(Long thesisId, Long sectionId) {
14         String key = "draft:" + thesisId + ":" + sectionId;
15         Object val = redisTemplate.opsForValue().get(key);
16         return val != null ? val.toString() : null;
17     }
18
19     public void deleteDraft(Long thesisId, Long sectionId) {
20         String key = "draft:" + thesisId + ":" + sectionId;
21         redisTemplate.delete(key);
22     }
23 }

```

### 16.2.3 JWT 令牌管理

用户登录成功后生成 JWT，将 Token 存储于 Redis 并与用户 ID 关联。用户主动登出或管理员强制下线时，将 Token 标记为失效。

**Listing 37:** 登录与登出的 Redis 操作

```

1 // 登录成功
2 String token = jwtUtil.generateToken(userId, role);
3 redisTemplate.opsForValue().set("token:" + userId, token,
4     Duration.ofHours(24));
5
6 // 登出
7 redisTemplate.delete("token:" + userId);
8 // Token 黑名单 (可选增强: 存储 jti 在黑名单中)
9 redisTemplate.opsForValue().set("blacklist:" + tokenId, "1",
10     Duration.ofHours(24));

```

JwtFilter 在校验每个请求时, 先从 Redis 查询 Token 是否在黑名单中。若存在, 即使 JWT 签名有效也拒绝该请求。Token 续期则通过独立的 Refresh Token (有效期 7 天) 进行, Refresh Token 同样存储于 Redis。

### 16.2.4 API 限流实现

基于 Redis 计数器实现登录接口的简单限流:

**Listing 38:** 基于 Redis 的登录限流实现

```

1 @Component
2 @RequiredArgsConstructor
3 public class RateLimiter {
4     private final RedisTemplate<String, Object> redisTemplate;
5
6     // 60 秒内同一 IP 最多登录 5 次
7     public boolean tryAcquire(String ip) {
8         String key = "rate:" + ip + ":login";
9         Long count = redisTemplate.opsForValue().increment(key);
10        if (count != null && count == 1) {
11            redisTemplate.expire(key, Duration.ofSeconds(60));
12        }
13        return count != null && count <= 5;
14    }
15 }

```

每次登录尝试调用 `tryAcquire(clientIp)`, 若返回 `false` 则拒绝请求并返回 HTTP 429 (Too Many Requests)。

### 16.2.5 缓存监控与运维

- **内存使用监控:** 通过 Redis `INFO memory` 命令定期采集 `used_memory_human` 指标, 设置 80% 告警阈值。
- **命中率统计:** 在 Service 层埋点记录缓存命中/未命中次数, 通过 Actuator 的 `/actuator/metrics` 暴露自定义指标 `cache.hit.ratio`。

- **慢查询日志：** Redis 配置 `slowlog-log-slower-than 10000`（超过 10ms 记录），定期巡检有无因大 Key 或复杂数据结构导致的慢操作。

## 16.3 部署详细设计

本节提供可直接使用的 Dockerfile、Docker Compose 编排文件及环境配置模板，确保在任何安装了 Docker 的 Linux 服务器上均可一键启动完整系统。

### 16.3.1 Dockerfile

采用多阶段构建（Multi-stage Build），第一阶段使用 Maven 编译打包，第二阶段使用精简 JRE 镜像运行，最终镜像体积控制在 200MB 以内。

**Listing 39:** 多阶段构建 Dockerfile

```

1 # ===== 构建阶段 =====
2 FROM maven:3.9-eclipse-temurin-21 AS builder
3 WORKDIR /build
4 COPY pom.xml .
5 RUN mvn dependency:go-offline -B
6 COPY src ./src
7 RUN mvn clean package -DskipTests -q
8
9 # ===== 运行阶段 =====
10 FROM eclipse-temurin:21-jre-alpine
11 WORKDIR /app
12 COPY --from=builder /build/target/*.jar app.jar
13 # 健康检查
14 HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
15   CMD wget -qO- http://localhost:8080/actuator/health || exit 1
16 EXPOSE 8080
17 ENTRYPOINT ["java", "-jar", "app.jar"]

```

### 16.3.2 Docker Compose 编排

以下编排文件定义了三个服务及其依赖关系、网络与数据卷。

**Listing 40:** docker-compose.yml

```

1 version: "3.9"
2 services:
3   postgres:
4     image: postgres:16-alpine
5     container_name: thesis-pg
6     environment:
7       POSTGRES_DB: thesis_generator
8       POSTGRES_USER: thesis_user
9       POSTGRES_PASSWORD: ${DB_PASSWORD:-thesis_pass}

```

```
10 volumes:
11   - pgdata:/var/lib/postgresql/data
12 networks:
13   - thesis-net
14 healthcheck:
15   test: ["CMD-SHELL", "pg_isready -U thesis_user -d thesis_generator"]
16   interval: 10s
17   timeout: 5s
18   retries: 5
19 restart: unless-stopped
20 deploy:
21   resources:
22     limits:
23       cpus: "1.0"
24       memory: 512M
25
26 redis:
27   image: redis:7-alpine
28   container_name: thesis-redis
29   command: redis-server --appendonly yes --maxmemory 256mb
30     --maxmemory-policy allkeys-lru
31   volumes:
32     - redisdata:/data
33   networks:
34     - thesis-net
35   healthcheck:
36     test: ["CMD", "redis-cli", "ping"]
37     interval: 10s
38     timeout: 5s
39     retries: 5
40   restart: unless-stopped
41   deploy:
42     resources:
43       limits:
44         cpus: "0.5"
45         memory: 256M
46
47 app:
48   build: .
49   container_name: thesis-app
50   ports:
51     - "8080:8080"
52   environment:
53     SPRING_PROFILES_ACTIVE: prod
54     SPRING_DATASOURCE_URL: jdbc:postgresql:
55       //postgres:5432/thesis_generator
56     SPRING_DATASOURCE_USERNAME: thesis_user
57     SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD:-thesis_pass}
```



```
58     SPRING_DATA_REDIS_HOST: redis
59     SPRING_DATA_REDIS_PORT: 6379
60     JWT_SECRET: ${JWT_SECRET}
61 depends_on:
62     postgres:
63         condition: service_healthy
64     redis:
65         condition: service_healthy
66 networks:
67     - thesis-net
68 restart: unless-stopped
69 deploy:
70     resources:
71         limits:
72             cpus: "2.0"
73             memory: 1G
74
75 volumes:
76     pgdata:
77     redisdata:
78
79 networks:
80     thesis-net:
81         driver: bridge
```

### 16.3.3 关键配置说明

1. **Redis 持久化：** `--appendonly yes` 启用 AOF (Append Only File) 持久化，确保 Redis 重启后缓存数据不丢失（用于恢复草稿与令牌状态）。 `--maxmemory 256mb --maxmemory-policy allkeys-lru` 设置内存上限并在满时自动淘汰最近最少使用的 Key。
2. **数据库密码：** 通过 `${DB_PASSWORD}` 环境变量注入，避免密码硬编码在配置文件中。启动时通过 `export DB_PASSWORD=xxx && docker-compose up -d` 或 `.env` 文件传入。
3. **JWT 密钥：** 同样通过环境变量 `${JWT_SECRET}` 注入，生产环境须更换为足够强度的随机字符串（建议 256 位以上）。
4. **健康检查：** PostgreSQL 使用 `pg_isready` 命令、Redis 使用 `redis-cli ping`、应用使用 Actuator health 端点。应用的 `depends_on` 设置为依赖 PostgreSQL 与 Redis 均 healthy 后才启动。
5. **网络隔离：** 仅应用容器的 8080 端口对外暴露，数据库与缓存的端口仅在内部 `thesis-net` 网络中可达，避免外部直接攻击数据存储层。

### 16.3.4 生产环境补充建议

- **反向代理**: 在 Docker Compose 中添加 Nginx 容器作为入口反向代理, 配置 HTTPS 终止、静态资源缓存与请求限流。
- **日志收集**: 应用日志 (通过 Logback) 输出到 stdout, 由 Docker 的 json-file 日志驱动收集, 再通过 Filebeat 或 Promtail 接入集中式日志平台 (ELK / Loki)。
- **监报告警**: 集成 Prometheus + Grafana, 通过 Spring Boot Actuator 的 /actuator/prometheus 端点暴露 JVM 指标、HTTP 请求指标与自定义缓存命中率指标。
- **数据库备份**: 为 PostgreSQL 数据卷配置定时备份脚本 (pg\_dump), 将备份文件上传至对象存储 (如阿里云 OSS / AWS S3)。
- **集群扩展**: 当单机性能不足时, 可将应用容器扩展为多副本 (docker-compose up --scale app=3), 前端加 Nginx 负载均衡; Redis 升级为哨兵模式或集群模式; PostgreSQL 采用主从复制读写分离。

### 16.3.5 部署操作手册

1. **环境准备**: 确保服务器已安装 Docker  $\geq 24.0$  与 Docker Compose  $\geq 2.20$ 。
2. **获取代码**: `git clone https://github.com/zzy154204-gif/thesis-generator.git && cd thesis-generator`。
3. **设置环境变量 (可选)**:

```
export DB_PASSWORD=your_secure_db_password
export JWT_SECRET=your_256bit_random_secret
```

4. **构建并启动**: `docker-compose up -d --build`。
5. **验证服务**:

```
curl http://localhost:8080/actuator/health
# 预期返回: {"status":"UP"}
```

6. **查看日志**: `docker-compose logs -f app`。
7. **停止服务**: `docker-compose down` (保留数据); `docker-compose down -v` (清除所有数据)。